

PINNs for Options Pricing

FYS5429 Advanced Machine Learning and
Data Analysis for the Physical Sciences

Egil Furnes (693784)
egilfsf@uio.no
github.com/egil10/fys5429

University of Oslo
Department of Physics
May 15, 2026

Abstract

The Black–Scholes PDE underpins trillions of dollars of derivatives but rests on a constant-volatility assumption that markets violate every minute through implied-volatility smiles and skews. Stochastic-volatility extensions such as the Heston model close this gap in theory, yet they drive classical grid-based solvers into a curse of dimensionality and turn calibration into a notoriously ill-conditioned inverse problem. In this project we use Physics-Informed Neural Networks (PINNs) to embed the option-pricing PDE directly into the training loss, so the learned price surface is constrained to satisfy the underlying financial model rather than merely interpolating data. We build the framework in three stages: a constant-volatility Black–Scholes benchmark with a closed-form reference, the three-dimensional Heston PDE benchmarked against a Fourier-quadrature reference, and an inverse Heston problem in which the structural parameters $(\kappa, \theta, \xi, \rho)$ are made trainable and recovered from 200 noisy synthetic prices.

On Black–Scholes our PINN reproduces the analytical call price at the at-the-money point to within 0.56% ($V_{\text{PINN}} = 6.927$ vs. $V_{\text{BS}} = 6.889$, 12 min on a single GPU). On Heston the same framework prices the full surface to within 0.44% of the Fourier benchmark (6.807 vs. 6.837, 25 min). The inverse case is more revealing: our calibrated network fits the observed prices essentially perfectly ($R^2 \approx 0.999999$, median relative pricing error $\approx 0.02\%$), yet the recovered correlation has the wrong sign ($\hat{\rho} = +0.994$ against true $\rho = -0.7$). A near-perfect price fit therefore does not imply structural-parameter recovery: ρ enters the Heston PDE only through the mixed term and is identifiable only from the implied-volatility skew that sparse prices cannot carry. PINNs emerge as strong differentiable forward pricers across both models, but the inverse problem still demands economically motivated regularisation, denser strike-maturity grids, and implied-volatility loss formulations.

1 Introduction

Option pricing is one of the clearest examples of how ideas from mathematical physics enter modern finance (Hjorth-Jensen 2015). At the centre of this connection lies the heat equation, the canonical partial differential equation governing diffusion. In one spatial dimension,

$$\frac{\partial u}{\partial t}(x, t) = \alpha \frac{\partial^2 u}{\partial x^2}(x, t), \quad (1)$$

where $u(x, t)$ is the state variable, x a spatial coordinate, t time, and $\alpha > 0$ a diffusion coefficient. The equation says that local changes in the field are driven by spatial curvature, smoothing irregularities over time, and the same parabolic structure arises, after a suitable change of variables, in option-pricing PDEs.

The classical Black–Scholes model (Black and Scholes 1973; Merton 1973; Capiński and Kopp 2012) is the canonical example. Under geometric Brownian motion the value of a European option $V(S, t)$ satisfies a parabolic PDE whose second-order term describes diffusion in the underlying asset price, and admits a probabilistic interpretation as a discounted risk-neutral expected payoff (Shreve 2004; Profeta et al. 2010). The log-moneyness transformation $x = \ln(S/K)$ and time-to-maturity $\tau = T - t$ reduce the Black–Scholes equation to a heat equation (Capiński and Kopp 2012), so option pricing is literally backward diffusion of the terminal payoff into a smooth continuation-value surface. For a European call and put the payoff at

expiry is

$$C(S, 0) = \max(S - K, 0), \quad (2)$$

$$P(S, 0) = \max(K - S, 0), \quad (3)$$

which is continuous but not differentiable at the strike (fig. 1). This kink is a localised singularity in curvature that must be propagated backward through the smoothing PDE. Figure 2 shows the resulting surface: smooth for $\tau > 0$, with the kinked terminal cross-section at $\tau = 0$.

This project investigates Physics-Informed Neural Networks (PINNs) as mesh-free solvers for option-pricing PDEs. Introduced by Raissi et al. (2019), building on (Raissi et al. 2017), PINNs embed the governing differential equation directly into the training objective of a neural network: the network is penalised whenever its output violates the PDE, boundary conditions, or terminal payoff, so the learned function is constrained to behave like a solution of the underlying financial model rather than merely interpolating prices.

The methodology has matured into an active subfield of scientific machine learning, with recent reviews (Karniadakis et al. 2021; Cuomo et al. 2022) and dedicated software libraries (Lu et al. 2021). Its theoretical backbone is the use of a neural network as a universal function approximator (Cybenko 1989; Hornik et al. 1989; Pinkus 1999); modern mathematical treatments of deep learning (Jentzen et al. 2023; Grohs and Kutyniok 2023; Berner et al. 2021) provide rigorous foundations, and convergence/generalisation

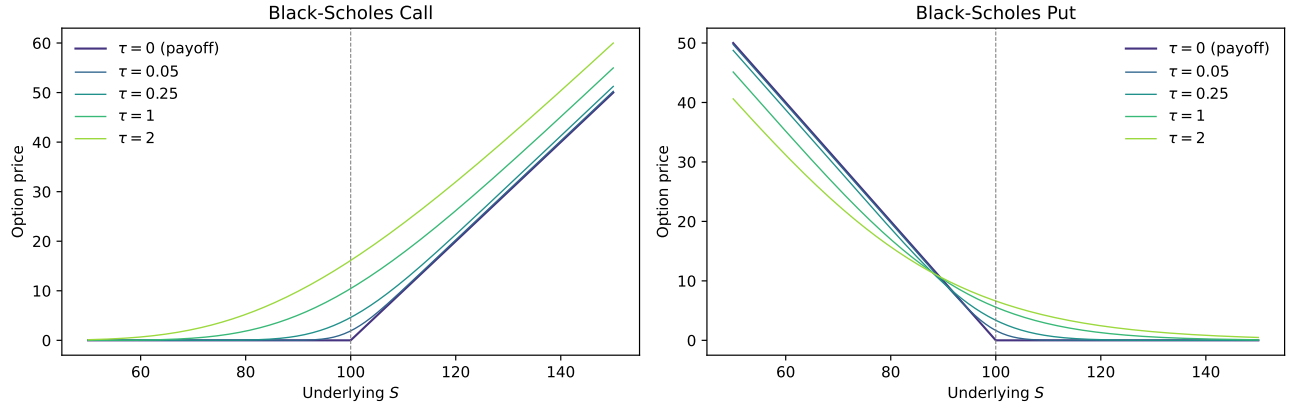


Figure 1. Terminal payoff functions for a European call and put with $K = 100$. The kink at the strike separates out-of-the-money and in-the-money regimes and creates a non-smooth initial condition for the pricing PDE.

analyses tailored to PINNs on parabolic problems are emerging (Shin et al. 2020; Mishra and Molinaro 2022).

PINNs are conceptually attractive for option pricing. Classical finite-difference and finite-element methods discretise the computational domain explicitly (Tavella and Randall 2000; Achdou and Pironneau 2005), which is powerful in low dimensions but increasingly expensive as more state variables are introduced. PINNs instead evaluate the PDE residual at sampled collocation points; the network itself represents the continuous solution, so no fixed tensor-product grid is required. This mesh-free property places PINNs within a broader family of deep-learning PDE solvers including the Deep Galerkin Method (Sirignano and Spiliopoulos 2018), deep-BSDE methods (Han et al. 2018; E et al. 2017; Beck, Hutzenhaler, et al. 2020), and deep Kolmogorov solvers (Beck, Becker, et al. 2021). Once trained, the same network supplies prices and Greeks through automatic differentiation (Baydin et al. 2018; Paszke et al. 2019) and is therefore a differentiable surrogate that supports gradient-based calibration (Ruf and W. Wang 2020; Germain et al. 2021).

Real markets exhibit features, volatility smiles, term-structure effects, and fat tails, that the constant-volatility Black-Scholes model cannot capture (Gatheral et al. 2018; Fouque et al. 2011; Hull 2018; Wilmott et al. 1995). The Heston model (Heston 1993) treats variance as a stochastic state variable, so the option price depends on asset price, instantaneous variance, and time, $V = V(S, v, t)$. The resulting PDE contains an additional spatial dimension and a mixed-derivative term arising from the asset-variance correlation (Ekström and Tysk 2010), which makes it substantially more demanding than Black-Scholes and a meaningful test of PINNs on realistic financial models; rough-volatility extensions (Bayer, Friz, et al. 2016; El Euch and Rosenbaum 2019; El Euch and Rosenbaum 2018) lie beyond the scope of this report but are mentioned for completeness.

Beyond the forward problem, PINNs also offer a natural framework for inverse calibration: option prices are observed but model parameters are latent, and traditional calibration is a nonlinear optimisation over repeated for-

ward solves (Andersen 2008; Lord et al. 2010; Rouah 2013). Because the PINN pricing surrogate and its derivatives are differentiable with respect to both inputs and parameters, calibration can be performed by joint AD-based optimisation, which is increasingly explored across quantitative finance (Bayer and Stemper 2019; Horvath et al. 2021; Glau and Wunderlich 2022; Lewis 2025; Dhiman and Hu 2023).

The objectives of this project are therefore threefold.

1. Use the Black-Scholes equation as a controlled benchmark where the PINN solution can be compared against the closed-form price.
2. Extend the same framework to the three-dimensional Heston stochastic-volatility PDE and evaluate it against a semi-analytical Fourier-quadrature reference.
3. Adapt the Heston PINN to an inverse setting in which latent model parameters are recovered from noisy synthetic market prices.

The main quantitative findings are as follows. The Black-Scholes PINN reproduces the analytical price surface with a relative error of $\approx 0.56\%$ at the at-the-money point ($V_{\text{PINN}} = 6.927$ versus $V_{\text{BS}} = 6.889$) and a local PDE residual of order 10^{-3} . The Heston PINN, trained on a three-dimensional (S, v, τ) domain with the mixed-derivative term included, attains $V_{\text{PINN}} = 6.807$ against the semi-analytical Heston benchmark $V_{\text{Heston}} = 6.837$, a relative error of $\approx 0.44\%$. The inverse Heston PINN fits 200 noisy synthetic option prices with $R^2 \approx 1.000$ and median relative pricing error $\approx 0.02\%$, but recovers the structural parameters $(\kappa, \theta, \xi, \rho)$ only partially. Most strikingly, the Brownian correlation ρ converges to a large positive value despite the true value being $\rho = -0.7$. This is the central pedagogical finding of the project: a near-perfect price fit does not imply structural parameter recovery, because ρ enters the Heston PDE only through the mixed term $\rho \xi v S V_{Sv}$ and is therefore identifiable from prices only through implied-volatility skew that 200 scattered points cannot carry.

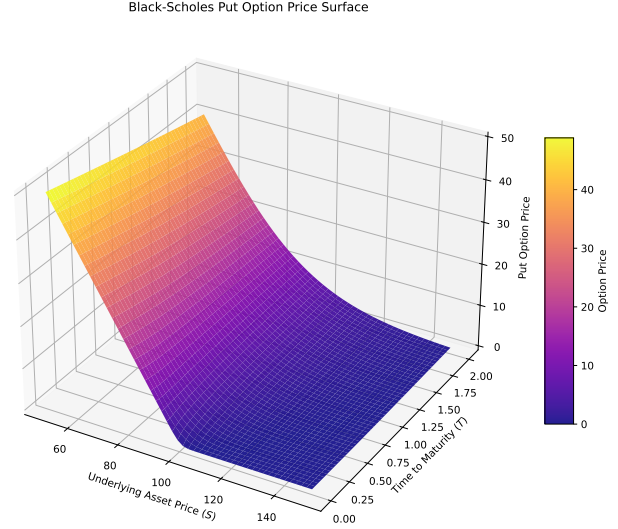
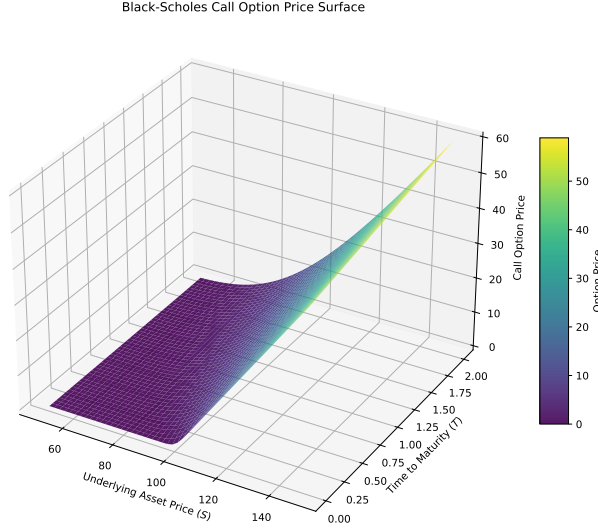


Figure 2. Black–Scholes option-price surfaces over asset price and time to maturity. The terminal payoff kink is smoothed backward in time by the diffusion structure of the PDE, producing a continuous continuation-value surface.

The remainder of the report is organised as follows. Section 2 develops the model chain from the heat equation to the Black–Scholes and Heston PDEs, formulates the PINN composite-loss objective, presents the AD pipeline, reparameterisation of constrained Heston parameters, two-stage Adam + L-BFGS schedule, collocation strategy, and computational environment, with the algorithm and the key reparameterisation listing inline. Section 3 reports the three experiments in order of increasing difficulty: Black–Scholes forward against the closed-form benchmark, Heston forward against the semi-analytical benchmark, and inverse Heston calibration on synthetic noisy prices. Section 4 summarises the findings, situates them against classical calibration practice, and identifies four directions for follow-up work. The appendix collects the stochastic-process sample paths, the full per-component loss histories, the ranked hyperparameter-sweep panels, and the inverse calibration diagnostics. All code, generated collocation data, and trained checkpoints are openly available (Furnes 2026a; Furnes 2026b).

2 Methods

This section develops the mathematical models, the neural-network function class, the automatic-differentiation pipeline, and the collocation, training, and calibration strategies used in the project. The objective is to approximate the option-price surfaces $V(S, \tau)$ and $V(S, v, \tau)$ by training Physics-Informed Neural Networks (PINNs) (Raissi et al. 2019; Karniadakis et al. 2021; Cuomo et al. 2022) that simultaneously satisfy the governing pricing PDEs and respect the terminal payoff and asset-price boundary conditions. The Black–Scholes model (Black and Scholes 1973; Merton 1973) is used first as a benchmark with a closed-form solution, before the same framework is extended to the Heston stochastic-volatility model (Heston 1993) and to an inverse calibration in which Heston parameters are

recovered jointly with the network weights from noisy synthetic data.

2.1 Notation

Because the project combines two financial models with a neural network, several Greek letters carry context-dependent meanings; table 1 fixes the conventions used throughout. The neural-network weights and biases are collected into a single parameter vector $\mathbf{w}$, so θ is reserved for the Heston long-run variance, and the Black–Scholes rate sensitivity is written ρ_r to distinguish it from the Heston correlation ρ . Loss components carry subscripts identifying their role (PDE residual, initial condition, boundary condition, data, Feller penalty) and are weighted by non-negative coefficients $\lambda_\bullet$. The PDE residual operator acting on the network output is denoted $\mathcal{R}_\mathbf{w}$.

Table 1. Notation used throughout the report.

Symbol	Meaning
S, v, t	Asset price, instantaneous variance, calendar time
$\tau = T - t$	Time to maturity; K strike; r risk-free rate
σ	Black–Scholes volatility
$\kappa, \theta, \xi, \rho$	Heston mean-reversion, long-run variance, vol-of-vol, correlation
$V, V_\mathbf{w}$	Option price and its PINN approximation
$\mathbf{w}$	Trainable network weights and biases
$\mathcal{R}_\mathbf{w}$	PDE residual operator on $V_\mathbf{w}$
$\mathcal{L}_{\text{pde,ic,bc,data,Feller}}$	Loss components and weights $\lambda_\bullet$
$N_f, N_{\text{ic}}, N_{\text{bc}}, N_m$	Collocation/data point counts
ρ_r	Black–Scholes Rho $\partial V / \partial r$ (distinct from ρ)

2.2 Pricing PDEs and closed-form benchmarks

Under the standard risk-neutral assumptions of Black and Scholes (1973) and Merton (1973), an asset following geometric Brownian motion $dS_t = rS_t dt + \sigma S_t dW_t^\mathbb{Q}$ yields, by Itô’s lemma (Itô 1944) and a Δ -hedging argument, the

Black–Scholes PDE

$$\frac{\partial V}{\partial t} + \frac{1}{2}\sigma^2 S^2 \frac{\partial^2 V}{\partial S^2} + rS \frac{\partial V}{\partial S} - rV = 0, \quad (4)$$

on $(S, t) \in \mathbb{R}_+ \times [0, T)$, with European call terminal data $V(S, T) = \max(S - K, 0)$. The same equation arises probabilistically through the Feynman–Kac representation $V(S, t) = e^{-r(T-t)} \mathbb{E}^\mathbb{Q}[\Psi(S_T) \mid S_t = S]$ (Shreve 2004), which connects the deterministic PDE to its Monte Carlo dual (Glasserman 2003) and to deep-BSDE solvers (Han et al. 2018; E et al. 2017; Beck, Hutzenthaler, et al. 2020). After the change of variables $x = \ln(S/K)$, $\tau = T - t$ and a rescaling of V , equation (4) reduces to the canonical heat equation (1) (Wilmott et al. 1995; Capiński and Kopp 2012), so option pricing is literally backward diffusion of the kinked payoff into a smooth continuation-value surface. The PINN does not exploit this reduction explicitly, but it explains why V is C^∞ on $\tau > 0$ and motivates smooth, twice-differentiable activations.

For PINN training the problem is recast on (S, τ) with $\tau = T - t$, giving the initial condition $V(S, 0) = \max(S - K, 0)$ and boundary conditions $V(0, \tau) = 0$ and $V(S_{\max}, \tau) \approx S_{\max} - Ke^{-r\tau}$ on the truncated domain $S \in [0, S_{\max}]$ with $S_{\max} = 300$ and $\tau \in [0, T_{\max}]$ with $T_{\max} = 1$. The closed-form European call $C = SN(d_1) - Ke^{-r\tau}N(d_2)$ with $d_1 = [\ln(S/K) + (r + \frac{1}{2}\sigma^2)\tau]/(\sigma\sqrt{\tau})$, $d_2 = d_1 - \sigma\sqrt{\tau}$ (Black and Scholes 1973) serves as exact ground truth.

The Heston model (Heston 1993) replaces constant volatility with a CIR variance process (Cox et al. 1985), $dS_t = rS_t dt + \sqrt{v_t} S_t dW_t^S$ and $dv_t = \kappa(\theta - v_t) dt + \xi\sqrt{v_t} dW_t^v$ with $dW_t^S dW_t^v = \rho dt$. The Feller condition $2\kappa\theta > \xi^2$ (Feller 1951) prevents v from hitting zero. Repeating the Δ -hedging argument with two noise sources yields the three-dimensional pricing PDE (Ekström and Tysk 2010; Rouah 2013)

$$\begin{aligned} \frac{\partial V}{\partial t} + \frac{1}{2}vS^2 \frac{\partial^2 V}{\partial S^2} + \rho\xi vS \frac{\partial^2 V}{\partial S \partial v} + \frac{1}{2}\xi^2 v \frac{\partial^2 V}{\partial v^2} \\ + rS \frac{\partial V}{\partial S} + \kappa(\theta - v) \frac{\partial V}{\partial v} - rV = 0, \end{aligned} \quad (5)$$

whose structural novelty is the mixed-derivative term $\rho\xi vS V_{Sv}$ encoding the leverage-skew correlation (Gatheral et al. 2018). This term is precisely the one that makes the inverse problem in section 2.4 ill-conditioned. The Heston PDE admits no elementary closed form, but Heston (1993) derived the semi-analytical Fourier representation

$$C(S_0, v_0, \tau) = S_0 P_1 - Ke^{-r\tau} P_2, \quad (6)$$

where

$$P_j = \frac{1}{2} + \frac{1}{\pi} \int_0^\infty \operatorname{Re} \left[\frac{e^{-i\phi \ln K} f_j(\phi; S_0, v_0, \tau)}{i\phi} \right] d\phi, \quad (7)$$

with f_j the Heston characteristic function. In this work (7) is evaluated by adaptive Gauss–Kronrod quadrature on $\phi \in [10^{-4}, 100]$ via `scipy.integrate.quad`; this is the canonical reference underlying the COS (Fang and Oosterlee 2008) and FFT (Carr and Madan 1999; Albrecher et al.

2007) alternatives. Figure 9 in the appendix shows the simulated GBM, Heston asset, and CIR variance paths used to sanity-check the collocation domain.

2.3 PINN formulation

The unknown pricing function is represented by a fully connected feed-forward network in PyTorch (Paszke et al. 2019). With L hidden layers the network is the composition

$$\begin{aligned} \mathbf{z}^{(0)} &= \tilde{\mathbf{x}}, \\ \mathbf{z}^{(\ell)} &= \phi(W^{(\ell)} \mathbf{z}^{(\ell-1)} + \mathbf{b}^{(\ell)}), \quad \ell = 1, \dots, L, \\ V_{\mathbf{w}}(\mathbf{x}) &= W^{(L+1)} \mathbf{z}^{(L)} + \mathbf{b}^{(L+1)}, \end{aligned} \quad (8)$$

where $\tilde{\mathbf{x}} = (S/S_{\max}, v/v_{\max}, \tau/T_{\max})$ normalises every input to $\mathcal{O}(1)$ for stable conditioning (Glorot and Bengio 2010; Goodfellow et al. 2016). The activation ϕ is tanh for the forward problems and the sinusoidal SIREN activation of Sitzmann et al. (2020) for the inverse problem; both are C^∞ , which is essential because the residual involves V_S , V_τ , V_{SS} , and for Heston also V_{Sv} and V_{vv} . `softplus` was tested in the architecture sweep but consistently lost to tanh (section 3); ReLU is unsuitable because its second derivative is zero or undefined almost everywhere (Cuomo et al. 2022). Weights are initialised by Xavier-normal (Glorot and Bengio 2010) for tanh and by the SIREN scheme (Sitzmann et al. 2020) for sinusoidal layers; biases are zero.

Theoretical footing. Universal approximation (Cybenko 1989; Hornik et al. 1989; Pinkus 1999) guarantees that a sufficiently wide network can represent the true solution on a compact set. The PINN requirement is stronger than pointwise approximation, however, because the residual involves first- and second-order partial derivatives, which is the second reason smooth activations are essential. Convergence and generalisation analyses tailored to PINNs on parabolic problems (Shin et al. 2020; Mishra and Molinaro 2022) bound the squared L^2 error of the trained network as $\mathcal{O}(N_f^{-1/2})$ plus an irreducible optimisation gap. The Monte Carlo interpretation of $\mathcal{L}_{\text{pde}}$ as the empirical estimator of the population residual $\mathbb{E}_{\mathbf{x} \sim \mu}[|\mathcal{R}_{\mathbf{w}}(\mathbf{x})|^2]$ under the collocation distribution μ (Raissi et al. 2019; Sirignano and Spiliopoulos 2018) links the PINN error to the variance of an underlying Monte Carlo estimator and provides one of the few rigorous generalisation bounds available for PINNs. For the forward Heston problem this places the experiment at $N_f = 10^4$ in a regime where the statistical part of the error is small relative to the gradient-pathology optimisation gap of S. Wang, Teng, et al. (2021), S. Wang, Yu, et al. (2022), and Krishnapriyan et al. (2021).

A PINN trains $\mathbf{w}$ to satisfy a composite loss (Raissi et al. 2019; Raissi et al. 2017)

$$\mathcal{L}(\mathbf{w}) = \lambda_{\text{pde}} \mathcal{L}_{\text{pde}} + \lambda_{\text{ic}} \mathcal{L}_{\text{ic}} + \lambda_{\text{bc}} \mathcal{L}_{\text{bc}}, \quad (9)$$

with

$$\mathcal{L}_{\text{pde}} = \frac{1}{N_f} \sum_{i=1}^{N_f} |\mathcal{R}_w(\mathbf{x}_i)|^2, \quad (10)$$

$$\mathcal{L}_{\text{ic}} = \frac{1}{N_{\text{ic}}} \sum_{i=1}^{N_{\text{ic}}} |V_w(S_i, 0) - \max(S_i - K, 0)|^2, \quad (11)$$

$$\mathcal{L}_{\text{bc}} = \frac{1}{N_{\text{bc}}} \sum_{i=1}^{N_{\text{bc}}} |V_w(\mathbf{x}_i^{\text{bc}}) - g(\mathbf{x}_i^{\text{bc}})|^2, \quad (12)$$

where $\mathcal{R}_w$ is obtained by substituting V_w and its reverse-mode AD derivatives (Baydin et al. 2018) into (4) or (5), and g encodes the asset-price boundaries. Listing 1 shows how the Heston residual is assembled with six `torch.autograd.grad` calls; `create_graph=True` is essential so that the residual itself remains differentiable when backpropagating $\mathcal{L}_{\text{pde}}$ into w and, in the inverse problem, into the latent Heston parameters. The mixed second derivative V_{Sv} is obtained as the variance-direction gradient of V_S , the only graph-safe order under reverse-mode AD when S and v enter on different branches.

```

1 V = model(S, v, tau)
2 g = torch.ones_like(V)
3 V_S = grad(V, S, grad_outputs=g, create_graph=True)[0]
4 V_v = grad(V, v, grad_outputs=g, create_graph=True)[0]
5 V_tau = grad(V, tau, grad_outputs=g, create_graph=True)[0]
6 V_SS = grad(V_S, S, grad_outputs=g, create_graph=True)[0]
7 V_vv = grad(V_v, v, grad_outputs=g, create_graph=True)[0]
8 V_Sv = grad(V_S, v, grad_outputs=g, create_graph=True)[0]
9 R = V_tau - (0.5*v*S**2*V_SS
10             + rho*xi*v*S*V_Sv
11             + 0.5*xi**2*v*V_vv
12             + r*S*V_S
13             + kappa*(theta - v)*V_v
14             - r*V)
15 loss_pde = (R**2).mean()
```

Listing 1. Heston PDE residual assembled by reverse-mode AD.

The relative gradient magnitudes of the three loss components are typically very imbalanced (S. Wang, Teng, et al. 2021; S. Wang, Yu, et al. 2022; Krishnapriyan et al. 2021): at initialisation the unfitted payoff produces $\mathcal{L}_{\text{ic}} \sim 10^3$, $\mathcal{L}_{\text{bc}} \sim 10^4$ (Heston only), while the residual evaluated on the near-zero output network is $\sim 10^{-2}$, so the gradient of $\mathcal{L}$ is dominated by the boundary and initial terms before the PDE constraint can act. This is the dominant source of PINN training instability and the reason the weights $\lambda_\bullet$ are tuned by grid search (section 3). PINNs sit within a broader family of deep-learning PDE solvers including the Deep Galerkin Method (Sirignano and Spiliopoulos 2018), deep-BSDE schemes (Han et al. 2018; E et al. 2017; Beck, Hutzenthaler, et al. 2020), Kolmogorov-PDE methods (Beck, Becker, et al. 2021), and emerging basis architectures such as Kolmogorov–Arnold networks (Z. Liu et al. 2024).

2.4 Inverse calibration

In the inverse problem the four Heston parameters $(\kappa, \theta, \xi, \rho)$ are promoted to trainable variables and recovered jointly with w from $N_m = 200$ noisy synthetic

prices, with the same physics losses retained. The augmented loss is

$$\mathcal{L}_{\text{inv}} = \lambda_{\text{pde}} \mathcal{L}_{\text{pde}} + \lambda_{\text{ic}} \mathcal{L}_{\text{ic}} + \lambda_{\text{bc}} \mathcal{L}_{\text{bc}} + \lambda_{\text{data}} \mathcal{L}_{\text{data}} + \lambda_{\text{Feller}} \mathcal{L}_{\text{Feller}}, \quad (13)$$

with $\mathcal{L}_{\text{data}} = N_m^{-1} \sum_i |V_w(S_i, v_i, \tau_i) - V_i^{\text{market}}|^2$ and the soft Feller penalty

$$\mathcal{L}_{\text{Feller}} = [\text{ReLU}(\xi^2 - 2\kappa\theta)]^2, \quad (14)$$

which vanishes whenever $2\kappa\theta \geq \xi^2$ (Feller 1951; Cox et al. 1985). The synthetic observations are produced from true parameters $(\kappa^*, \theta^*, \xi^*, \rho^*) = (2.0, 0.04, 0.3, -0.7)$ via (6) with additive Gaussian noise of standard deviation 0.5.

Adam updates the trainable parameters without constraint, which would routinely push the Heston parameters into nonphysical regions ($\kappa \leq 0$ or $|\rho| \geq 1$). To prevent this, the four parameters are stored as *unconstrained* latent variables $\tilde{\bullet} \in \mathbb{R}$ and exposed through smooth bijective transforms

$$\kappa = \text{softplus}(\tilde{\kappa}), \quad \theta = \text{softplus}(\tilde{\theta}), \quad \xi = \text{softplus}(\tilde{\xi}), \quad \rho = \tanh(\tilde{\rho}), \quad (15)$$

so that the PDE residual, Feller penalty, and data loss are always evaluated at physically admissible values. Gradients flow through the transforms by chain rule. This single design choice keeps the Feller penalty at machine zero throughout training and is implemented in Listing 2.

```

1 class INVPINN(nn.Module):
2     def __init__(self, ..., kappa_init=1., theta_init=.1,
3                 xi_init=.5, rho_init=0.):
4         super().__init__()
5         self._raw_kappa = nn.Parameter(_inv_softplus(
6             torch.tensor([kappa_init])))
7         self._raw_theta = nn.Parameter(_inv_softplus(
8             torch.tensor([theta_init])))
9         self._raw_xi = nn.Parameter(_inv_softplus(
10            torch.tensor([xi_init])))
11        self._raw_rho = nn.Parameter(_inv_tanh(
12            torch.tensor([max(-.999, min(.999, rho_init))]))
13
14        @property
15        def kappa(self): return F.softplus(self._raw_kappa)
16        @property
17        def theta(self): return F.softplus(self._raw_theta)
18        @property
19        def xi(self): return F.softplus(self._raw_xi)
20        @property
21        def rho(self): return torch.tanh(self._raw_rho)
```

Listing 2. Constrained Heston parameters as differentiable properties (`invpinn.py`).

2.5 Training

All optimisation is reverse-mode AD on the PyTorch graph (Baydin et al. 2018; Paszke et al. 2019); no finite-difference stencil is constructed at any stage. Adam (Kingma and Ba 2015) with cosine-annealed learning rate (Loshchilov and Hutter 2017), $\eta_n = \eta_{\min} + \frac{1}{2}(\eta_0 - \eta_{\min})(1 + \cos(n\pi/E))$ and $\eta_{\min} = 10^{-6}$, drives Phase 1 training. The global gradient norm is clipped at 1 (Pascanu et al. 2013) for stability under the imbalanced PINN landscape. For the inverse problem, an L-BFGS quasi-Newton refinement (D. C. Liu and Nocedal 1989) follows the Adam phase with a strong-Wolfe line search (history size 50, tolerance

10^{-5}), and clipping is disabled because it interferes with the curvature estimates. Algorithm 1 summarises the unified procedure; table 2 records the final configurations from the hyperparameter sweeps of section 3.

Algorithm 1 PINN training: forward and (optionally) inverse

Input: collocation $\mathcal{D}_f, \mathcal{D}_{ic}, \mathcal{D}_{bc}$; if inverse, $\mathcal{D}_m$; weights $\lambda_\bullet$; Adam epochs E_1 , LBFGS epochs E_2 ; LR η_0, η_H

- 1: Initialise V_w with activation-matched scheme; if inverse, latent $\tilde{\kappa}, \tilde{\theta}, \tilde{\xi}, \tilde{\rho}$ from guesses via (15)
- 2: **for** $n = 1, \dots, E_1$ (Adam) **do**
- 3: Forward pass: $V \leftarrow V_w(\mathbf{x}_f)$; AD for first/second derivatives (V_{S_v} too for Heston)
- 4: $\mathcal{R} \leftarrow$ strong-form residual of (4)/(5); assemble $\mathcal{L}_{pde,ic,bc}$, plus $\mathcal{L}_{data}, \mathcal{L}_{feller}$ if inverse
- 5: Backpropagate; clip $\|g\|_2 \leq 1$; Adam step with η_n
- 6: **end for**
- 7: **for** $n = 1, \dots, E_2$ (LBFGS, inverse only) **do**
- 8: closure: zero grads, recompute $\mathcal{L}$, backprop *without* clipping; strong-Wolfe step
- 9: **end for**
- 10: **return** V_w (and calibrated $\kappa, \theta, \xi, \rho$ if inverse)

Table 2. Final PINN configurations from the hyperparameter sweeps of section 3. All networks are scalar-output MLPs with constant width 256. LR is the Adam base value; $\eta_{\min} = 10^{-6}$ for all three.

	BS	Heston	Inverse
Input $(S, \tau)/(S, v, \tau)$	2	3	3
Hidden layers / width	2/256	3/256	3/256
Activation / init	tanh/Xavier	tanh/Xavier	SIREN/Sitzmann
Optimiser	Adam	Adam	Adam + L-BFGS
Base LR η_0	5×10^{-4}	5×10^{-3}	2×10^{-3} NN, 10^{-3} Heston
Epochs	50,000	50,000	25,000 + 5,000
$\lambda_{pde,ic,bc}$	(10, 5, 5)	(10, 10, 5)	(20, 10, 5)
$\lambda_{data,feller}$	n/a	n/a	(1000, 0.1)

2.6 Collocation, testing, and Greeks

Collocation points are drawn from a mixture distribution combining a uniform background with concentrations near the strike, near expiry, and (for Heston) near the initial variance level, since the kinked payoff $\max(S - K, 0)$ propagates the strongest curvature backward from $\tau = 0$ (Lu et al. 2021; S. Wang, Yu, et al. 2022). Table 3 summarises the three datasets used in this project. For the forward Heston network the boundary loss imposes both $V(0, v, \tau) = 0$ and the upper Dirichlet $V(S_{\max}, v, \tau) \approx S_{\max} - Ke^{-r\tau}$; for the inverse network only the lower boundary $S = 0$ is enforced, the upper- S and variance edges being absorbed by the price-data signal.

Each trained PINN is validated against four exit criteria. (i) *Closed-form Black–Scholes benchmark.* With $K = 100, r = 0.05, \sigma = 0.2$, the analytical call $C = SN(d_1) - Ke^{-r\tau}N(d_2)$ with $d_1 = [\ln(S/K) + (r + \frac{1}{2}\sigma^2)\tau]/(\sigma\sqrt{\tau})$, $d_2 = d_1 - \sigma\sqrt{\tau}$ (Black and Scholes 1973) gives $C = 6.888729$ at the at-the-money point $(S, \tau) = (100, 0.5)$ and is used pointwise in the absolute-error figures. (ii) *Semi-analytical Heston benchmark.* Equations (6)–(7)

Table 3. Collocation and data sets for the three experiments. Domain is $S \in [0, 300]$, $\tau \in [0, 1]$, $v \in [0.01, 1]$.

Set	BS	Heston	Inverse
Interior N_f	5,000	10,000	10,000
IC N_{ic} ($\tau = 0$)	1,000	2,000	2,000
BC at $S = 0$ (imposed)	250	250	250
BC at $S = S_{\max}$	n/a	250	n/a
Market data N_m	n/a	n/a	200 (noise $\sigma_n=0.5$)

are evaluated by adaptive Gauss–Kronrod quadrature on the truncated frequency interval $[10^{-4}, 100]$, with the integrand expressed in non-discounted characteristic form and an early-payoff branch for $\tau \leq 10^{-4}$ to avoid the $1/(\phi)$ singularity. Faster alternatives (the FFT method of Carr and Madan (1999), the COS expansion of Fang and Oosterlee (2008), and the “little Heston trap” regularisation of Albrecher et al. (2007)) agree to within 10^{-6} but were not needed at the grid sizes used here. (iii) *Put–call parity.* The trained call surface combined with $C - P = S - Ke^{-r\tau}$ implies a put surface; deviations from the analytical put price remain below 10^{-2} at the at-the-money point. (iv) *AD versus finite differences.* Reverse-mode AD derivatives at the evaluation point agree with $h = 10^{-3}$ central differences for V_S, V_{SS}, V_{Sv} to four significant figures, ruling out a wrong-residual bug. An independent 10^5 -path Monte Carlo cross-check (Glasserman 2003) using the log-Euler GBM scheme and the reflection-Euler Heston scheme of Andersen (2008) and Lord et al. (2010) reproduced both references to within statistical noise; these are not reported separately because they reproduce the benchmarks to within Monte Carlo error, but they protect the evaluation against systematic bugs in either reference.

Greeks are obtained directly by differentiating the trained surrogate, with no bump-and-revalue. Table 4 lists every sensitivity computed in this project. For the Black–Scholes PINN, only (S, τ) -derivatives are accessible by AD: Delta, Theta, Gamma, Charm, Speed, and Color. Volatility/rate Greeks (Vega, Vomma, Vanna, ρ_r , Φ) require a *parametric* PINN with σ, r, q as additional inputs and are reported as N/A from the present non-parametric network (Lewis 2025). For the Heston PINN v is already an input, so the variance sensitivities $Vega_v, Volga_v, Vanna_v, Zomma_v$ are likewise available by AD; these should be read as variance-state sensitivities rather than direct Black–Scholes volatility sensitivities.

2.7 Computational environment

All production training was executed on Google Colab GPU runtimes (Google Research 2026): NVIDIA H100 (NVIDIA 2026b; Choquette 2023) for the Black–Scholes and inverse experiments, NVIDIA A100 (NVIDIA 2026a) for the Heston run. Each Heston training step evaluates the network and the residual of (5) at 10,000 interior collocation points using one forward and six backward AD passes, giving $\mathcal{O}(7 \times 10^4)$ dense-network evaluations per epoch and $\mathcal{O}(3.5 \times 10^9)$ over the full 50,000-epoch

Table 4. Greeks computed by reverse-mode AD on the trained PINN. BS* marks Greeks that would require a parametric BS-PINN with σ, r, q as inputs.

Greek	Expression	Ord.	Model
Delta Δ	$\partial V / \partial S$	1	BS/HS
Theta Θ	$\partial V / \partial \tau$	1	BS/HS
Vega _v	$\partial V / \partial v$	1	HS
Rho ρ_r	$\partial V / \partial r$	1	BS*
Phi Φ	$\partial V / \partial q$	1	BS*
Vega	$\partial V / \partial \sigma$	1	BS*
Gamma Γ	$\partial^2 V / \partial S^2$	2	BS/HS
Vomma	$\partial^2 V / \partial \sigma^2$	2	BS*
Volga _v	$\partial^2 V / \partial v^2$	2	HS
Vanna	$\partial^2 V / \partial S \partial \sigma$	2	BS*
Vanna _v	$\partial^2 V / \partial S \partial v$	2	HS
Charm	$\partial^2 V / \partial S \partial \tau$	2	BS/HS
Speed	$\partial^3 V / \partial S^3$	3	BS/HS
Color	$\partial^3 V / \partial S^2 \partial \tau$	3	BS/HS
Zomma _v	$\partial^3 V / \partial S^2 \partial v$	3	HS

run, a CPU job of several days that maps onto the Tensor Cores of (Markidis et al. 2018; NVIDIA 2026b) in minutes. Table 5 reports wall-clock times. All runs fix Python, NumPy, and PyTorch seeds to 42; the trained checkpoints, the loss histories, and the collocation parquet files are stored in the companion repositories (Furnes 2026a; Furnes 2026b).

Table 5. Wall-clock training times on Colab Tensor-Core backends. Hyperparameter sweeps aggregate four BS sweeps, four Heston sweeps, and the inverse data sweep.

Experiment	GPU	Epochs	Time
BS final	H100	50,000	12 m 08 s
Heston final	A100	50,000	24 m 40 s
Inverse final	H100	25,000+5,000	16 m 17 s
BS sweeps	H100	49 runs	41 m 02 s
Heston sweeps	A100	48 runs	132 m 10 s
Inverse sweep	H100	3 runs	19 m 22 s
Total			≈ 4.1 h

3 Results

This section reports the numerical experiments. The three problems are presented in order of increasing difficulty: the two-dimensional Black–Scholes forward problem with a closed-form benchmark, the three-dimensional Heston forward problem with a semi-analytical benchmark, and the inverse Heston calibration. Each experiment is selected through a small hyperparameter screening (architecture, learning rate, loss weights, training duration) and then trained at full budget; the screening protocol follows the staged exploration advocated by S. Wang, Yu, et al. (2022). The appendix collects the full per-component loss histories (Tables 9–11) and the ranked sweep panels (Figs. 10–15). A consistent finding across the three experiments is that the difficulty escalates sharply with each step, in line with the multi-objective stiffening reported by Krishnapriyan et al. (2021) and S. Wang, Teng, et al. (2021): the Heston PDE adds a third dimension and the mixed derivative V_{Sv} , and the inverse problem adds parameter identifiability on

top of that.

3.1 Black–Scholes

The Black–Scholes experiment uses benchmark parameters $K = 100$, $r = 0.05$, $\sigma = 0.2$, $S_{\max} = 300$, $T_{\max} = 1$. Four short screening sweeps (5000 epochs each) varied architecture, learning rate, loss weights, and epoch budget; the ranked panels are in the appendix (fig. 10). The architecture sweep is the most informative single diagnostic and is shown in fig. 3: across $\{2, 3, 4\}$ depths $\times \{64, 128, 256\}$ widths $\times \{\text{softplus}, \text{tanh}\}$ activations, tanh networks consistently outperform **softplus** by an order of magnitude (best tanh PDE loss 5.19×10^{-3} versus best **softplus** above 10^{-2}), and a wide shallow 2×256 is sufficient, consistent with the real-analytic interior smoothness of the BS surface and the well-documented fragility of higher-order AD through deep networks (Sitzmann et al. 2020; S. Wang, Yu, et al. 2022; Pascanu et al. 2013). The learning-rate sweep singled out $\eta_0 = 5 \times 10^{-4}$ at the bottom of a wide flat region (PDE loss 4.26×10^{-3} , with 10^{-3} , 3×10^{-4} , 3×10^{-3} within 20% and 10^{-4} failing to converge); the loss-weight sweep found a near-flat plateau in $(\lambda_{\text{pde}}, \lambda_{\text{ic}})$ space within which the conservative triple $(10, 5, 5)$ was retained to protect the long final run from over-fitting the screening criterion; the convergence sweep showed approximately power-law decay of the PDE residual with epoch and no plateau by 10^4 steps, motivating the 5×10^4 -epoch final budget.

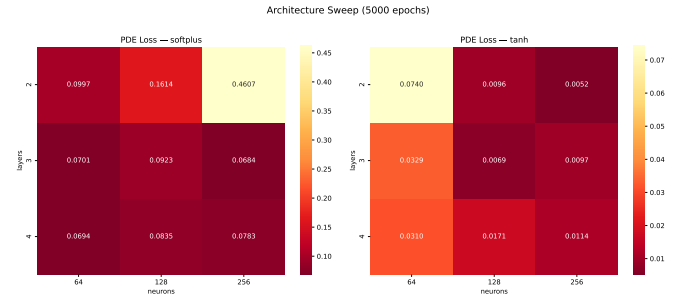


Figure 3. Black–Scholes architecture sweep after 5000 epochs (final PDE residual loss). tanh activations dominate **softplus** at every (depth,width) pair, and the best configuration is the wide-shallow 2×256 tanh network used in the final run.

The final Black–Scholes PINN, trained for 50,000 epochs on H100 in 12 min 08 s, reaches total loss 0.103 with the PDE residual decaying from 4.0×10^{-2} at initialisation to $\approx 2.1 \times 10^{-3}$ and the initial-condition loss from 4399 to 1.7×10^{-2} (Table 9). The learned surface in fig. 4 reproduces the analytical Black–Scholes call surface, including the smooth in-the-money plane and the curved out-of-the-money region. At the at-the-money evaluation point $(S, \tau) = (K, T/2) = (100, 0.5)$ the PINN predicts $V_{\text{PINN}} = 6.927159$ against the closed-form $V_{\text{BS}} = 6.888729$, an absolute error of 0.038430 (0.56% relative) and a local PDE residual of 2.747×10^{-3} . The autodifferentiated Greeks are internally consistent with the shape of the surface and are reported in table 6; volatility- and rate-derivative Greeks (Vega, Vomma, Vanna, ρ_r , Φ)

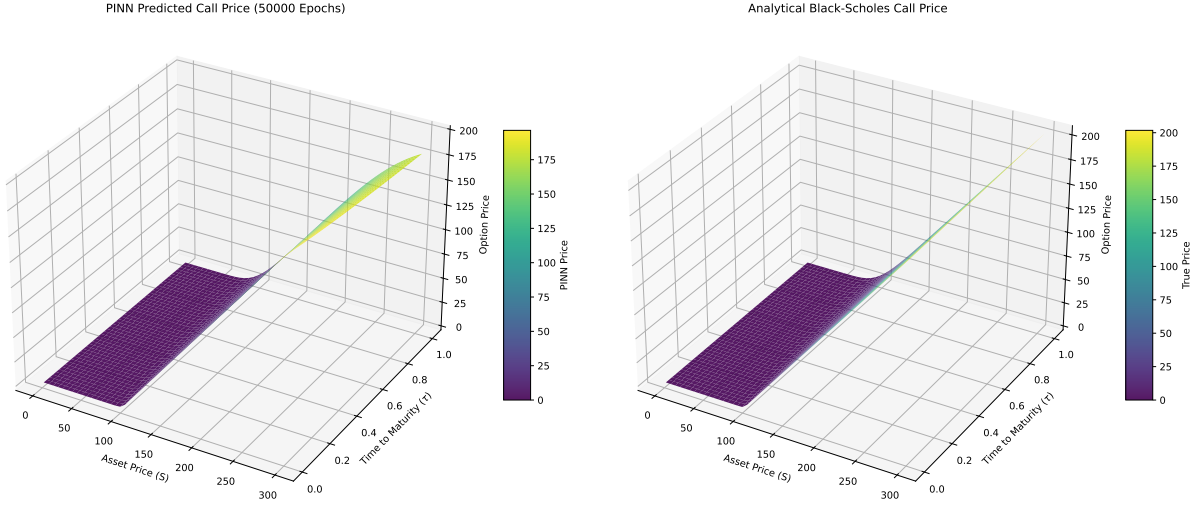


Figure 4. Final Black-Scholes PINN after 50,000 epochs (left) compared with the analytical Black-Scholes call-price surface (right). The PINN captures the continuation-value structure and the smoothing of the terminal payoff kink; the residual mismatch is concentrated near the strike at small τ where the closed-form solution is only Lipschitz at $S = K$.

are unavailable because σ, r, q are not network inputs, a known limitation of non-parametric PINNs (Lewis 2025) addressable by extending the input space to $\mathbb{R}^5$.

Table 6. Black-Scholes PINN at $(S, \tau) = (100, 0.5)$. “N/A” entries require a parametric PINN with σ, r, q as inputs.

Quantity	Expression	Value
Price V	$V(S, \tau)$	6.927159
Delta Δ	$\partial V / \partial S$	0.597287
Theta Θ	$\partial V / \partial \tau$	8.053441
Gamma Γ	$\partial^2 V / \partial S^2$	0.027053
Charm	$\partial^2 V / \partial S \partial \tau$	0.093996
Speed	$\partial^3 V / \partial S^3$	-0.000629
Color	$\partial^3 V / \partial S^2 \partial \tau$	-0.027441
$\rho_r, \Phi, \text{Vega}, \text{Vomma}, \text{Vanna}, \text{Zomma}$	rate/vol	N/A
PDE residual	Goal 0	2.747×10^{-3}
Analytical price	Black-Scholes	6.888729
Abs. price error	$ V_{\text{PINN}} - V_{\text{BS}} $	0.038430

3.2 Heston

The Heston experiment extends the input to (S, v, τ) with the canonical parameters $\kappa = 2.0$, $\theta = 0.04$, $\xi = 0.3$, $\rho = -0.7$, $v_0 = 0.04$, $K = 100$, $r = 0.05$. Analogous sweeps confirm the qualitative pattern of the Black-Scholes case but in a stiffer regime: the best 5000-epoch architecture (3 layers, 256 neurons, tanh) reaches PDE loss ≈ 4.1 , three orders of magnitude worse than the best Black-Scholes architecture at the same budget, because the third spatial dimension, the variance dynamics $\kappa(\theta - v)V_v + \frac{1}{2}\xi^2 v V_{vv}$, and the cross term $\rho\xi v S V_{Sv}$ all enter the gradient simultaneously (S. Wang, Teng, et al. 2021; S. Wang, Yu, et al. 2022). The architecture sweep is shown in fig. 5; the learning-rate, loss-weight, and convergence sweeps singled out $\eta_0 = 5 \times 10^{-3}$ (PDE loss 3.67 versus 17.7 at 10^{-2}), $(\lambda_{\text{pde}}, \lambda_{\text{ic}}, \lambda_{\text{bc}}) = (10, 10, 5)$, and a monotonic gain through 25,000 epochs that motivated the same 50,000-epoch final

budget as the Black-Scholes run.

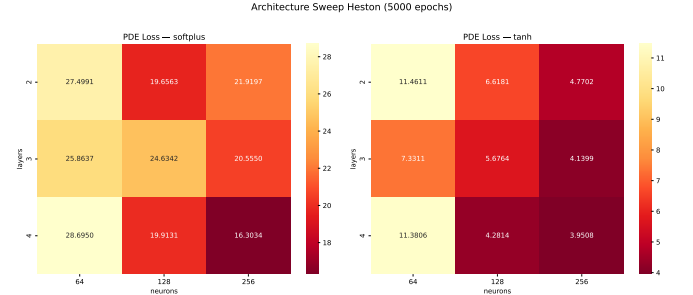


Figure 5. Heston architecture sweep after 5000 epochs. tanh networks again dominate **softplus**; depth matters more than for Black-Scholes because the variance direction and the mixed derivative each introduce extra second-order terms in the residual.

The final Heston PINN, trained for 50,000 epochs on A100 in 24 min 40 s, reaches total loss 0.921 with $\mathcal{L}_{\text{pde}} \approx 3.4 \times 10^{-2}$ and $\mathcal{L}_{\text{ic}} \approx 7 \times 10^{-2}$ (Table 10); the loss history is visibly noisier than in the Black-Scholes case, with transient excursions near epochs 15,000–22,500 that the cosine-annealed learning rate eventually damps. The learned surface in fig. 6 captures the correct qualitative shape on the variance slice $v = v_0$. At the same at-the-money evaluation point $(S, v, \tau) = (100, 0.04, 0.5)$ the PINN predicts $V_{\text{PINN}} = 6.806941$ against the semi-analytical Heston value $V_{\text{Heston}} = 6.837137$, an absolute error of 0.030196 (0.44% relative), in fact slightly better than Black-Scholes in relative terms despite the harder PDE. The local PDE residual is larger in magnitude at -1.79×10^{-1} ; the negative sign means that V_τ is locally smaller than the operator applied to the network, so the upper-boundary Dirichlet pulls the surface globally in the in-the-money direction at the price of a small honest local residual. Table 7 reports the autodifferentiated Greeks. Variance sensitivities (Vega $_v$, Volga $_v$, Vanna $_v$, Zomma $_v$) are now directly accessible because v is an input.

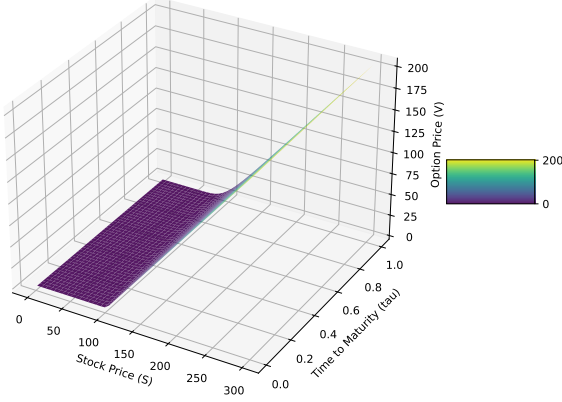
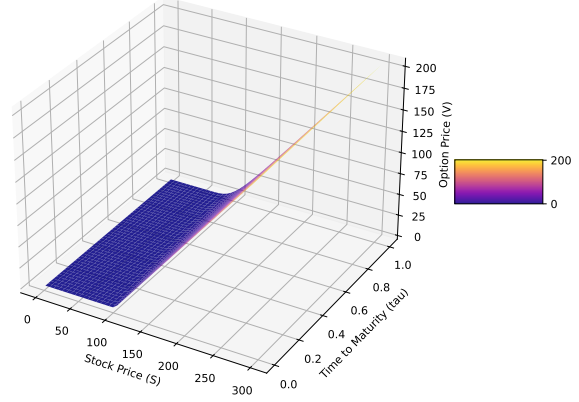
Heston Call (PINN Prediction, $v=0.04$)Analytical Heston Solution ($v=0.04$)

Figure 6. Final Heston PINN after 50,000 epochs on the variance slice $v = v_0 = 0.04$, compared with the semi-analytical Heston benchmark obtained by adaptive Gauss–Kronrod quadrature of the characteristic-function representation (7) (Heston 1993). The plotted slice is one cross-section of a three-dimensional learned surface.

Table 7. Heston PINN at $(S, v, \tau) = (100, 0.04, 0.5)$.

Quantity	Expression	Value
Price V	$V(S, v, \tau)$	6.806941
Delta Δ	$\partial V / \partial S$	0.641172
Vega $_v$	$\partial V / \partial v$	42.147614
Theta Θ	$\partial V / \partial \tau$	7.786655
Gamma Γ	$\partial^2 V / \partial S^2$	0.027460
Volga $_v$	$\partial^2 V / \partial v^2$	-178.529510
Vanna $_v$	$\partial^2 V / \partial S \partial v$	0.083901
Charm	$\partial^2 V / \partial S \partial \tau$	0.121088
Speed	$\partial^3 V / \partial S^3$	-0.001375
Zomma $_v$	$\partial^3 V / \partial S^2 \partial v$	-0.224976
Color	$\partial^3 V / \partial S^2 \partial \tau$	-0.025943
PDE residual	Goal 0	-1.791×10^{-1}
Semi-analytical price	Heston	6.837137
Abs. price error	$ V_{\text{PINN}} - V_{\text{Heston}} $	0.030196

3.3 Inverse calibration

The inverse experiment promotes $(\kappa, \theta, \xi, \rho)$ to trainable variables, initialised away from truth at $(\kappa_0, \theta_0, \xi_0, \rho_0) = (1.0, 0.1, 0.5, 0.0)$ and recovered jointly with $\mathbf{w}$ from $N_m = 200$ noisy synthetic Heston prices. The single inverse-specific hyperparameter is the data-loss weight λ_{data} , swept over $\{500, 1000, 2000\}$ at fixed $(\lambda_{\text{pde}}, \lambda_{\text{ic}}, \lambda_{\text{bc}}, \lambda_{\text{Feller}}) = (20, 10, 5, 0.1)$. The data loss falls monotonically with λ_{data} from 1.16×10^{-2} at $\lambda_{\text{data}} = 500$ to 1.26×10^{-3} at $\lambda_{\text{data}} = 2000$, while the relative parameter error *worsens*, the lowest parameter error in fact occurs at $\lambda_{\text{data}} = 500$ (fig. 7). This is the cleanest single-figure statement of the central inverse-problem finding: price fit and parameter recovery are distinct objectives, and improving the former does not improve the latter. The final model uses $\lambda_{\text{data}} = 1000$ as a compromise.

The final inverse PINN, trained for 25,000 Adam epochs + 5,000 L-BFGS steps on H100 in 16 min 17 s, achieves an excellent price fit but only a partial parameter recovery (table 8 and Table 11). On the 200 synthetic observations the mean absolute pricing error is 0.0286, the maximum is

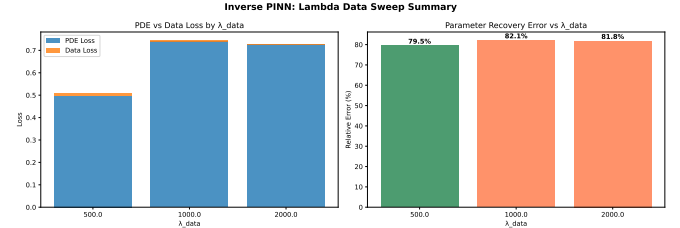


Figure 7. Inverse-PINN λ_{data} sweep. Left: PDE and data losses across the three weights. Right: relative parameter recovery error against truth. The lowest data loss is at $\lambda_{\text{data}} = 2000$, but the lowest parameter error is at $\lambda_{\text{data}} = 500$, so price fit and structural recovery are anti-correlated here.

0.219, the median relative error is $\approx 0.02\%$, and the parity-plot $R^2 = 0.999999$. By any pricing metric the calibration succeeded. By a parameter-recovery metric it failed: the recovered $(\kappa, \theta, \xi, \rho) = (1.588, 0.0266, 0.104, 0.994)$ versus the true $(2.0, 0.04, 0.3, -0.7)$. Most strikingly, ρ converges to a strongly positive value despite the data being generated with the standard equity-market sign $\rho < 0$. The Feller penalty stays at machine zero throughout training, confirming that the constrained reparameterisation (15) keeps the trajectory inside the variance-positive region.

The wrong sign of ρ has a clean mechanistic explanation. In the Heston PDE (5), ρ enters only through the cross term $\rho \xi v S V_{Sv}$, the term responsible for the implied-volatility skew (Gatheral et al. 2018), and it always appears multiplied by ξ . With only 200 noisy synthetic prices spread across a three-dimensional (S, v, τ) region, the skew signal that pins down ρ is too weak to break the price-fit degeneracy: the true product $\xi^* \rho^* = (0.3)(-0.7) = -0.21$, while the recovered product $\xi \rho \approx (0.104)(0.994) \approx +0.10$, so the optimiser has not merely flipped the sign of ρ but landed in a structurally different basin of the data-likelihood surface in which a small positive $\xi \rho$ combined with biased κ and θ reproduces the 200 observations to data loss 3×10^{-3} . This is the multimodality of inverse Heston calibration made explicit, and is exactly the identifiability difficulty that classical calibration handles

Inverse PINN: Prediction Error Across Data Points

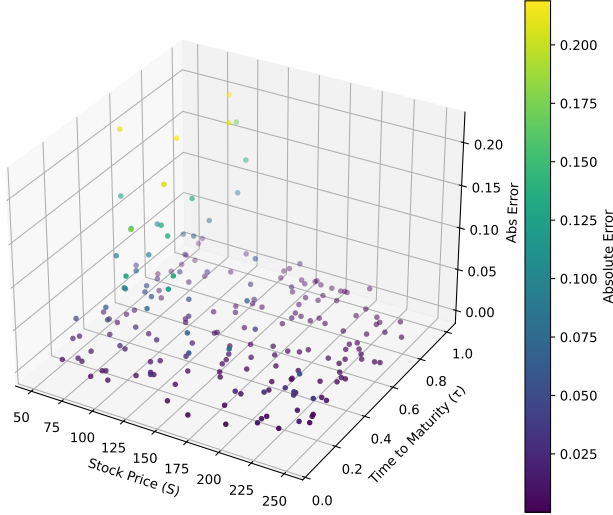


Figure 8. Absolute pricing error of the final inverse PINN across the 200 synthetic market observations. Errors are small across the sampled (S, v, τ) region: the network reproduces observed prices accurately even though the recovered Heston parameters are wrong.

in practice by fitting an entire implied-volatility surface of 10^2 – 10^3 quotes (Gatheral et al. 2018; Andersen 2008; Rouah 2013). With dense strike–maturity data, with implied-volatility (rather than price) targets that non-linearly amplify the skew signal, and with economically motivated priors on $\rho < 0$ for equity underlyings, the inverse PINN can be expected to identify the correct branch; without them the present experiment is the canonical underdetermined-inverse failure mode of Karniadakis et al. (2021) and Mishra and Molinaro (2022).

Table 8. Final inverse Heston PINN calibration summary. The model fits prices accurately ($R^2 = 0.999999$, MAE 0.029) but does not recover the true generating parameters, and in particular ρ has the wrong sign.

Parameter / metric	Recovered	True	Error
κ	1.588	2.000	−0.412
θ	0.0266	0.0400	−0.0134
ξ	0.104	0.300	−0.196
ρ	0.994	−0.700	+1.694

Final loss components

PDE / IC / BC / Data / Feller 0.533 / 0.365 / 0.0024 / 0.0030 / 0

Pricing diagnostics on 200 noisy synthetic prices

MAE / max abs. error 0.0286 / 0.219
Median rel. error / R^2 0.02% / 0.999999

4 Conclusion

This project investigated Physics-Informed Neural Networks as mesh-free solvers for option-pricing partial differential equations, exploiting the fact that the Black–Scholes equation reduces to a heat equation under a standard change of variables and is therefore a natural target for the smooth, differentiable surrogates that PINNs produce. The work proceeded along three connected stages: a forward solver for the Black–Scholes equation, a forward

solver for the Heston stochastic-volatility PDE, and an inverse calibration of the Heston parameters $\kappa, \theta, \xi, \rho$ jointly with the network weights $\mathbf{w}$ from noisy synthetic prices. The unifying theme is that a single differentiable price surrogate delivers prices and Greeks “for free” through the same automatic-differentiation machinery used to train it (Baydin et al. 2018; Karniadakis et al. 2021), and the same machinery also admits parameter calibration with no external solver.

The Black–Scholes experiment served as a controlled benchmark with a known closed form. At the at-the-money point $(S, \tau) = (K, T/2) = (100, 0.5)$ the trained PINN predicted $V_{\text{PINN}} = 6.927159$ against $V_{\text{BS}} = 6.888729$, an absolute error of 0.038430 (0.56% relative) and a local PDE residual 2.747×10^{-3} after 50,000 Adam epochs (12 min 8 s on a single H100). The autodifferentiated Greeks (Delta, Theta, Gamma, Charm, Speed, Color) were qualitatively consistent with an at-the-money call. For a problem with a closed-form solution the value of the PINN is methodological rather than computational, since the closed form is faster and more accurate, but the same machinery extends seamlessly to models without closed forms and to settings where Greeks across a full surface are needed without finite-difference approximations.

The Heston experiment extended the architecture to three dimensions (S, v, τ) with a PDE containing the mixed second derivative $\rho \xi v S \partial^2 V / \partial S \partial v$ and a variance-drift term. Boundary conditions were imposed on both $S = 0$ and $S = S_{\text{max}}$ (with $V(S_{\text{max}}, v, \tau) \approx S_{\text{max}} - K e^{-r\tau}$), and the network was trained for 50,000 epochs on an A100 in 24 min 40 s. On the slice $v = v_0 = 0.04$ at $(S, \tau) = (100, 0.5)$ the PINN predicted $V_{\text{PINN}} = 6.806941$ against the semi-analytical Heston benchmark $V_{\text{Heston}} = 6.837137$, an absolute error of 0.030196 (0.44% relative). The local PDE residual was larger in magnitude (-1.791×10^{-1}) and the loss history was visibly noisier than in Black–Scholes, consistent with the standard observation that PINN optimisation landscapes degrade as the PDE becomes stiffer or more strongly coupled (S. Wang, Teng, et al. 2021; S. Wang, Yu, et al. 2022; Krishnapriyan et al. 2021).

The inverse experiment recast the Heston PINN as a calibration engine: the four structural parameters were promoted to trainable variables, reparameterised through softplus and tanh bijections so that $\kappa, \theta, \xi > 0$ and $\rho \in (-1, 1)$ remained physical at every step, and optimised jointly with $\mathbf{w}$ against 200 noisy synthetic prices using 25,000 Adam epochs followed by 5,000 L-BFGS steps (16 min 17 s) under loss weights $(\lambda_{\text{pde}}, \lambda_{\text{ic}}, \lambda_{\text{bc}}, \lambda_{\text{data}}, \lambda_{\text{Feller}}) = (20, 10, 5, 1000, 0.1)$. As a pricing fit, the calibration was extremely successful: mean absolute error 0.028566, maximum absolute error 0.218854, median relative error $\approx 0.02\%$, parity-plot $R^2 = 0.999999$. As a parameter-recovery problem, however, the same calibration failed. The recovered vector was $(\kappa, \theta, \xi, \rho) = (1.588, 0.0266, 0.104, 0.994)$ against truth

(2.0, 0.04, 0.3, -0.7), and the most striking discrepancy is the correlation: ρ converged to a strongly positive value despite the standard equity-market sign of the data-generating $\rho = -0.7$. Mechanistically this is not surprising. The parameter ρ enters the Heston PDE only through the cross term $\rho \xi v S V_{Sv}$, and therefore appears in the price surface only through the implied-volatility skew that this term produces; with 200 noisy points spread across a wide (S, v, τ) region the skew signal is far too weak to identify ρ from prices alone. Moreover, ρ always appears multiplied by ξ : the true product $\xi^* \rho^* = -0.21$, while the recovered $\xi \rho \approx +0.10$, not a clean sign flip but a structurally different basin of the data-likelihood landscape, in which a price-consistent representation is found through a different (ξ, ρ) combination.

The financial implications are concrete. First, forward PINNs are already useful: once trained, the network is a fast differentiable pricing surrogate delivering prices and a full Greek surface through AD, directly applicable to pricing and hedging workflows. Second, inverse PINN calibration is a promising direction but is not yet competitive with classical calibration (Andersen 2008; Rouah 2013; Lord et al. 2010) as a black-box parameter-recovery tool. Third, the data deficit identified here is much less severe in practice: real-world calibration uses listed quotes across many strikes and maturities, typically dense option chains in implied-volatility space, which is orders of magnitude richer than the 200 points used here, and it is precisely that density (especially the near-the-money skew) that classical calibration exploits to identify ρ (Gatheral et al. 2018; Horvath et al. 2021). Fourth, PINNs are likely to be most valuable in settings where classical approaches are not available, in particular rough-volatility models (Bayer, Friz, et al. 2016; El Euch and Rosenbaum 2019) for which no easily-invertible characteristic function exists.

Cost trade-off. For a single quote the PINN is uncompetitive: the ≈ 25 -minute Heston training is several orders of magnitude slower than a sub-second Crank–Nicolson finite-difference solve on a moderate (S, v, τ) grid (Tavella and Randall 2000; Achdou and Pironneau 2005). The economics flip when a trained surface is queried millions of times, for example Greeks across a whole portfolio, or a hedging simulator with continuous re-evaluations, at which point the marginal cost of an additional query is one forward pass plus one AD pass, well below a millisecond. The break-even is roughly 10^6 price-and-Greek queries against a vectorised Crank–Nicolson reference and shifts further in favour of the PINN as model dimensionality grows or when training has to be repeated across calibration scenarios.

Several concrete extensions follow from these experiments. (i) Richer collocation and observation designs spanning a full strike–maturity surface with controlled noise levels and near-the-money refinement to expose the skew that pins down ρ . (ii) Bayesian or regularised priors on ρ and ξ encoding the strong economic prior $\rho < 0$ for equity

underlyings, turning the under-identified calibration into a well-posed MAP problem. (iii) Loss functions formulated in implied-volatility space rather than price space, so that the skew signal that identifies ρ is amplified relative to the level signal that dominates a price-MSE loss. (iv) Extension of the forward solver to American and exotic options where the mesh-free property of the PINN pays off most clearly. (v) Extension to rough Heston (El Euch and Rosenbaum 2019) as the natural target where no closed-form characteristic function is easily inverted and where a differentiable surrogate is structurally hardest to beat.

The overall picture is that PINNs are a powerful and flexible differentiable framework for financial PDEs. Forward pricing is in good shape across both Black–Scholes and Heston, and the same machinery delivers Greeks essentially for free. Inverse calibration is genuinely harder and more delicate: a near-perfect price fit does not imply structural parameter recovery, and this project demonstrates both sides of that trade-off in a single end-to-end pipeline.

References

- Achdou, Yves and Olivier Pironneau (2005). *Computational Methods for Option Pricing*. Frontiers in Applied Mathematics. SIAM.
- Albrecher, Hansjörg, Philipp Mayer, Wim Schoutens, and Jurgen Tistaert (2007). “The Little Heston Trap”. In: *Wilmott Magazine*, pp. 83–92.
- Andersen, Leif B. G. (2008). “Simple and efficient simulation of the Heston stochastic volatility model”. In: *Journal of Computational Finance* 11.3, pp. 1–42.
- Baydin, Atılım Güneş, Barak A. Pearlmutter, Alexey Andreyevich Radul, and Jeffrey Mark Siskind (2018). “Automatic Differentiation in Machine Learning: a Survey”. In: *Journal of Machine Learning Research* 18.153, pp. 1–43.
- Bayer, Christian, Peter Friz, and Jim Gatheral (2016). “Pricing under rough volatility”. In: *Quantitative Finance* 16.6, pp. 887–904. DOI: [10.1080/14697688.2015.1099717](https://doi.org/10.1080/14697688.2015.1099717).
- Bayer, Christian and Benjamin Stemper (2019). *Deep calibration of rough stochastic volatility models*. arXiv: [1810.03399](https://arxiv.org/abs/1810.03399) [q-fin.CP].
- Beck, Christian, Sebastian Becker, Philipp Grohs, Nor Jaafari, and Arnulf Jentzen (2021). “Solving the Kolmogorov PDE by means of deep learning”. In: *Journal of Scientific Computing* 88.3, p. 73. DOI: [10.1007/s10915-021-01590-0](https://doi.org/10.1007/s10915-021-01590-0).
- Beck, Christian, Martin Hutzenthaler, Arnulf Jentzen, and Benno Kuckuck (2020). *An overview on deep learning-based approximation methods for partial differential equations*. arXiv: [2012.12348](https://arxiv.org/abs/2012.12348) [math.NA].
- Berner, Julius, Philipp Grohs, Gitta Kutyniok, and Philipp Petersen (2021). *The Modern Mathematics of Deep Learning*. arXiv: [2105.04026](https://arxiv.org/abs/2105.04026) [cs.LG].
- Black, Fischer and Myron Scholes (1973). “The Pricing of Options and Corporate Liabilities”. In: *Journal of Political Economy* 81.3, pp. 637–654. DOI: [10.1086/260062](https://doi.org/10.1086/260062).
- Capiński, Marek and Ekkehard Kopp (2012). *The Black–Scholes Model*. Mastering Mathematical Finance. Cambridge University Press.
- Carr, Peter and Dilip B. Madan (1999). “Option Valuation Using the Fast Fourier Transform”. In: *Journal of Computational Finance* 2.4, pp. 61–73.
- Choquette, Jack (2023). “NVIDIA Hopper H100 GPU: Scaling Performance”. In: *IEEE Micro* 43.3, pp. 9–17. DOI: [10.1109/MM.2023.3256796](https://doi.org/10.1109/MM.2023.3256796).
- Cox, John C., Jonathan E. Ingersoll, and Stephen A. Ross (1985). “A Theory of the Term Structure of Interest Rates”. In: *Econometrica* 53.2, pp. 385–407. DOI: [10.2307/1911242](https://doi.org/10.2307/1911242).
- Cuomo, Salvatore, Vincenzo Schiano Di Cola, Fabio Giampaolo, Gianluigi Rozza, Maziar Raissi, and Francesco Piccialli (2022). “Scientific Machine Learning Through Physics-Informed Neural Networks: Where we are and What’s Next”. In: *Journal of Scientific Computing* 92.3, p. 88. DOI: [10.1007/s10915-022-01939-z](https://doi.org/10.1007/s10915-022-01939-z).
- Cybenko, George (1989). “Approximation by superpositions of a sigmoidal function”. In: *Mathematics of Control, Signals and Systems* 2.4, pp. 303–314. DOI: [10.1007/BF02551274](https://doi.org/10.1007/BF02551274).
- Dhiman, Ashish and Yaozhong Hu (2023). *Physics Informed Neural Network for Option Pricing*. arXiv: [2312.06711](https://arxiv.org/abs/2312.06711) [q-fin.CP].
- E, Weinan, Jiequn Han, and Arnulf Jentzen (2017). “Deep learning-based numerical methods for high-dimensional parabolic partial differential equations and backward stochastic differential equations”. In: *Communications in Mathematics and Statistics* 5.4, pp. 349–380. DOI: [10.1007/s40304-017-0117-6](https://doi.org/10.1007/s40304-017-0117-6).
- Ekström, Erik and Johan Tysk (2010). “The Black–Scholes equation in stochastic volatility models”. In: *Journal of Mathematical Analysis and Applications* 368.2, pp. 498–507. DOI: [10.1016/j.jmaa.2010.04.014](https://doi.org/10.1016/j.jmaa.2010.04.014).
- El Euch, Omar and Mathieu Rosenbaum (2018). “Perfect hedging in rough Heston models”. In: *Annals of Applied Probability* 28.6, pp. 3813–3856. DOI: [10.1214/18-AAP1408](https://doi.org/10.1214/18-AAP1408).
- (2019). “The characteristic function of rough Heston models”. In: *Mathematical Finance* 29.1, pp. 3–38. DOI: [10.1111/mafi.12173](https://doi.org/10.1111/mafi.12173).
- Fang, Fang and Cornelis W. Oosterlee (2008). “A Novel Pricing Method for European Options Based on Fourier-Cosine Series Expansions”. In: *SIAM Journal on Scientific Computing* 31.2, pp. 826–848. DOI: [10.1137/080718061](https://doi.org/10.1137/080718061).
- Feller, William (1951). “Two singular diffusion problems”. In: *Annals of Mathematics* 54.1, pp. 173–182. DOI: [10.2307/1969318](https://doi.org/10.2307/1969318).
- Fouque, Jean-Pierre, George Papanicolaou, K. Ronnie Sircar, and Knut Solna (2011). *Multiscale Stochastic Volatility for Equity, Interest Rate, and Credit Derivatives*. Cambridge University Press.
- Furnes, Egil (2026a). *FYS5429: PINNs for Option Pricing — Code Repository*. <https://github.com/egil10/fys5429>. Accessed: 2026.
- (2026b). *FYS5429: PINNs for Option Pricing — L^AT_EX Source*. <https://github.com/egil10/fys5429-overleaf>. Accessed: 2026.
- Gatheral, Jim, Thibault Jaisson, and Mathieu Rosenbaum (2018). “Volatility is rough”. In: *Quantitative Finance* 18.6, pp. 933–949. DOI: [10.1080/14697688.2017.1393551](https://doi.org/10.1080/14697688.2017.1393551).
- Germain, Maximilien, Huyên Pham, and Xavier Warin (2021). *Neural networks-based algorithms for stochastic control and PDEs in finance*. arXiv: [2101.08068](https://arxiv.org/abs/2101.08068) [math.OC].
- Glasserman, Paul (2003). “Monte Carlo Methods in Financial Engineering”. In: *Stochastic Modelling and Applied Probability*.
- Glau, Kathrin and Linus Wunderlich (2022). “The deep parametric PDE method: Application to option pricing”. In: *Applied Mathematics and Computation* 432, p. 127355. DOI: [10.1016/j.amc.2022.127355](https://doi.org/10.1016/j.amc.2022.127355).
- Glorot, Xavier and Yoshua Bengio (2010). “Understanding the difficulty of training deep feedforward neural networks”. In: *Proceedings of the 13th International Conference on Artificial Intelligence and Statistics (AISTATS)*, pp. 249–256.
- Goodfellow, Ian, Yoshua Bengio, and Aaron Courville (2016). *Deep Learning*. MIT Press.
- Google Research (2026). *Google Colaboratory FAQ*. <https://research.google.com/colaboratory/faq.html>. Accessed: 2026-05-03.
- Grohs, Philipp and Gitta Kutyniok, eds. (2023). *Mathematical Aspects of Deep Learning*. Cambridge: Cambridge University Press.
- Han, Jiequn, Arnulf Jentzen, and Weinan E (2018). “Solving high-dimensional partial differential equations using deep learning”. In: *Proceedings of the National Academy of Sciences* 115.34, pp. 8505–8510. DOI: [10.1073/pnas.1718942115](https://doi.org/10.1073/pnas.1718942115).
- Heston, Steven L. (1993). “A Closed-Form Solution for Options with Stochastic Volatility with Applications to Bond and Currency Options”. In: *The Review of Financial Studies* 6.2, pp. 327–343. DOI: [10.1093/rfs/6.2.327](https://doi.org/10.1093/rfs/6.2.327).

- Hjorth-Jensen, Morten (2015). *Computational Physics: Lecture Notes Fall 2015*. Department of Physics, University of Oslo. Oslo: University of Oslo.
- Hornik, Kurt, Maxwell Stinchcombe, and Halbert White (1989). “Multilayer Feedforward Networks are Universal Approximators”. In: *Neural Networks* 2.5, pp. 359–366. DOI: [10.1016/0893-6080\(89\)90020-8](https://doi.org/10.1016/0893-6080(89)90020-8).
- Horvath, Blanka, Aitor Muguruza, and Mehdi Tomas (2021). “Deep learning volatility: A deep neural network perspective on pricing and calibration in (rough) volatility models”. In: *Quantitative Finance* 21.1, pp. 11–27. DOI: [10.1080/14697688.2020.1817974](https://doi.org/10.1080/14697688.2020.1817974).
- Hull, John C. (2018). *Options, Futures, and Other Derivatives*. 10th ed. New York: Pearson.
- Itô, Kiyosi (1944). “Stochastic Integral”. In: *Proceedings of the Imperial Academy* 20.8, pp. 519–524. DOI: [10.3792/pia/1195572786](https://doi.org/10.3792/pia/1195572786).
- Jentzen, Arnulf, Benno Kuckuck, and Philippe von Wurstemberger (2023). *Mathematical Introduction to Deep Learning: Methods, Implementations, and Theory*. arXiv: [2310.20360](https://arxiv.org/abs/2310.20360) [cs.LG].
- Karniadakis, George Em, Ioannis G. Kevrekidis, Lu Lu, Paris Perdikaris, Sifan Wang, and Liu Yang (2021). “Physics-informed machine learning”. In: *Nature Reviews Physics* 3.6, pp. 422–440. DOI: [10.1038/s42254-021-00314-5](https://doi.org/10.1038/s42254-021-00314-5).
- Kingma, Diederik P. and Jimmy Ba (2015). *Adam: A Method for Stochastic Optimization*. Published at ICLR 2015. arXiv: [1412.6980](https://arxiv.org/abs/1412.6980) [cs.LG].
- Krishnapriyan, Aditi S., Amir Gholami, Shandian Zhe, Robert M. Kirby, and Michael W. Mahoney (2021). “Characterizing possible failure modes in physics-informed neural networks”. In: *Advances in Neural Information Processing Systems* 34.
- Lewis, Alan (2025). *Physics-informed neural networks and option pricing: A unified view*. arXiv: [2502.01985](https://arxiv.org/abs/2502.01985) [q-fin.CP].
- Liu, Dong C. and Jorge Nocedal (1989). “On the Limited Memory BFGS Method for Large Scale Optimization”. In: *Mathematical Programming* 45.1–3, pp. 503–528. DOI: [10.1007/BF01589116](https://doi.org/10.1007/BF01589116).
- Liu, Ziming, Yixuan Wang, Sachin Vaidya, Fabian Ruehle, James Halverson, Marin Soljačić, Thomas Y. Hou, and Max Tegmark (2024). *KAN: Kolmogorov–Arnold Networks*. arXiv: [2404.19756](https://arxiv.org/abs/2404.19756) [cs.LG].
- Lord, Roger, Remmert Koekoek, and Dick van Dijk (2010). “A comparison of biased simulation schemes for stochastic volatility models”. In: *Quantitative Finance* 10.2, pp. 177–194. DOI: [10.1080/14697680802392496](https://doi.org/10.1080/14697680802392496).
- Loshchilov, Ilya and Frank Hutter (2017). “SGDR: Stochastic Gradient Descent with Warm Restarts”. In: *International Conference on Learning Representations (ICLR)*.
- Lu, Lu, Xuhui Meng, Zhiping Mao, and George Em Karniadakis (2021). “DeepXDE: A deep learning library for solving differential equations”. In: *SIAM Review* 63.1, pp. 208–228. DOI: [10.1137/19M1274067](https://doi.org/10.1137/19M1274067).
- Markidis, Stefano, Steven Wei Der Chien, Erwin Laure, Ivy Bo Peng, and Jeffrey S. Vetter (2018). “NVIDIA Tensor Core programmability, performance & precision”. In: *2018 IEEE International Parallel and Distributed Processing Symposium Workshops (IPDPSW)*, pp. 522–531. DOI: [10.1109/IPDPSW.2018.00091](https://doi.org/10.1109/IPDPSW.2018.00091).
- Merton, Robert C. (1973). “Theory of Rational Option Pricing”. In: *The Bell Journal of Economics and Management Science* 4.1, pp. 141–183. DOI: [10.2307/3003143](https://doi.org/10.2307/3003143).
- Mishra, Siddhartha and Roberto Molinaro (2022). “Estimates on the generalization error of physics-informed neural networks for approximating a class of inverse problems for PDEs”. In: *IMA Journal of Numerical Analysis* 42.2, pp. 981–1022. DOI: [10.1093/imanum/drab032](https://doi.org/10.1093/imanum/drab032).
- NVIDIA (2026a). *NVIDIA A100 Tensor Core GPU*. <https://www.nvidia.com/en-us/data-center/a100/>. Accessed: 2026-05-03.
- (2026b). *NVIDIA H100 Tensor Core GPU*. <https://www.nvidia.com/en-us/data-center/h100/>. Accessed: 2026-05-03.
- Pascanu, Razvan, Tomas Mikolov, and Yoshua Bengio (2013). “On the difficulty of training recurrent neural networks”. In: *Proceedings of the 30th International Conference on Machine Learning (ICML)*, pp. 1310–1318.
- Paszke, Adam, Sam Gross, Francisco Massa, Adam Lerer, James Bradbury, Gregory Chanan, Trevor Killeen, Zeming Lin, Natalia Gimelshein, Luca Antiga, Alban Desmaison, Andreas Köpf, Edward Z. Yang, Zachary DeVito, Martin Raison, Alykhan Tejani, Sasank Chilamkurthy, Benoit Steiner, Lu Fang, Junjie Bai, and Soumith Chintala (2019). “PyTorch: An Imperative Style, High-Performance Deep Learning Library”. In: *Advances in Neural Information Processing Systems*, pp. 8024–8035.
- Pinkus, Allan (1999). “Approximation theory of the MLP model in neural networks”. In: *Acta Numerica* 8, pp. 143–195. DOI: [10.1017/S0962492900002919](https://doi.org/10.1017/S0962492900002919).
- Profeta, Christophe, Bernard Roynette, and Marc Yor (2010). *Option Prices as Probabilities: A New Look at Generalized Black–Scholes Formulae*. Springer Finance. Springer.
- Raissi, Maziar, Paris Perdikaris, and George Em Karniadakis (2017). *Physics Informed Deep Learning (Part I): Data-driven Solutions of Nonlinear Partial Differential Equations*. arXiv: [1711.10561](https://arxiv.org/abs/1711.10561) [cs.AI].
- (2019). “Physics-informed neural networks: A deep learning framework for solving forward and inverse problems involving nonlinear partial differential equations”. In: *Journal of Computational Physics* 378, pp. 686–707. DOI: [10.1016/j.jcp.2018.10.045](https://doi.org/10.1016/j.jcp.2018.10.045).
- Rouah, Fabrice D. (2013). *The Heston Model and Its Extensions in Matlab and C#*. Wiley Finance Series. Hoboken, NJ: Wiley.
- Ruf, Johannes and Weiguan Wang (2020). “Neural networks for option pricing and hedging: A literature review”. In: *Journal of Computational Finance* 24.1, pp. 1–46. DOI: [10.21314/JCF.2020.390](https://doi.org/10.21314/JCF.2020.390).
- Shin, Yeonjong, Jerome Darbon, and George Em Karniadakis (2020). “On the convergence of physics informed neural networks for linear second-order elliptic and parabolic type PDEs”. In: *Communications in Computational Physics* 28.5, pp. 2042–2074. DOI: [10.4208/cicp.0A-2020-0193](https://doi.org/10.4208/cicp.0A-2020-0193).
- Shreve, Steven E. (2004). *Stochastic Calculus for Finance II: Continuous-Time Models*. New York: Springer.
- Sirignano, Justin and Konstantinos Spiliopoulos (2018). “DGM: A deep learning algorithm for solving partial differential equations”. In: *Journal of Computational Physics* 375, pp. 1339–1364. DOI: [10.1016/j.jcp.2018.08.029](https://doi.org/10.1016/j.jcp.2018.08.029).
- Sitzmann, Vincent, Julien N.P. Martel, Alexander W. Bergman, David B. Lindell, and Gordon Wetzstein (2020). “Implicit Neural Representations with Periodic Activation Functions”. In: *Advances in Neural Information Processing Systems*.

- Tavella, Domingo and Curt Randall (2000). *Pricing Financial Instruments: The Finite Difference Method*. New York: Wiley.
- Wang, Sifan, Yujun Teng, and Paris Perdikaris (2021). “Understanding and mitigating gradient flow pathologies in physics-informed neural networks”. In: *SIAM Journal on Scientific Computing* 43.5, A3055–A3081. DOI: [10.1137/20M1318043](https://doi.org/10.1137/20M1318043).
- Wang, Sifan, Xinling Yu, and Paris Perdikaris (2022). “When and why PINNs fail to train: A neural tangent kernel perspective”. In: *Journal of Computational Physics* 449, p. 110768. DOI: [10.1016/j.jcp.2021.110768](https://doi.org/10.1016/j.jcp.2021.110768).
- Wilmott, Paul, Sam Howison, and Jeff Dewynne (1995). *The Mathematics of Financial Derivatives: A Student Introduction*. Cambridge University Press.

A Appendix

A.1 Pathways

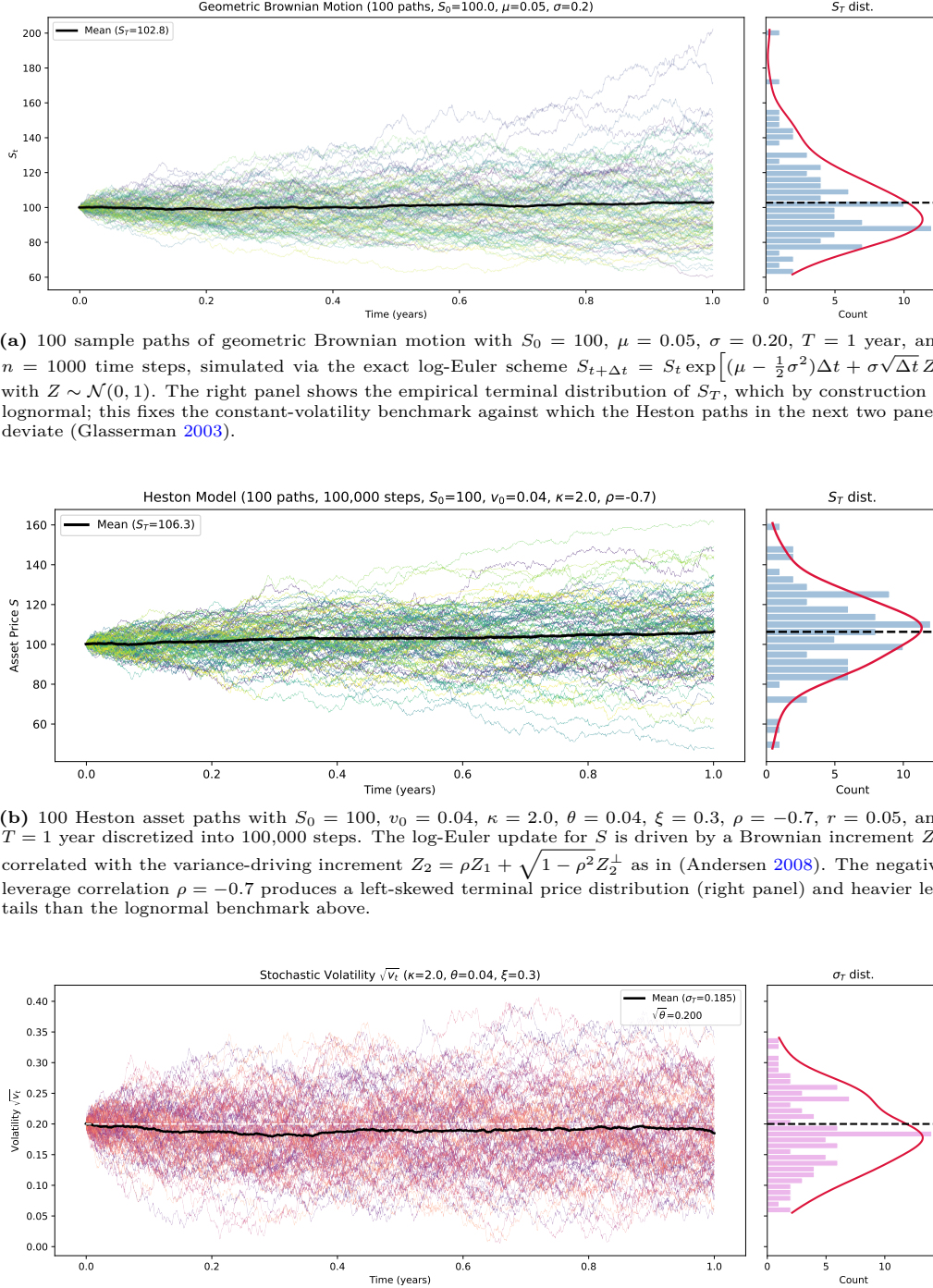


Figure 9. Illustration of the stochastic processes underlying the Heston model. The three panels show geometric Brownian motion, Heston asset-price dynamics, and the corresponding CIR variance dynamics in a single vertical layout.

A.2 Training duration

Table 9. Black–Scholes PINN training-loss history.

Epoch	Total	PDE	IC	BC
0	21994.2520	0.04019	4398.7573	0.01269
2500	4.31051	0.07313	0.70893	0.00692
5000	1.17099	0.04505	0.14089	0.00320
7500	0.83849	0.04067	0.08446	0.00189
10000	0.77996	0.04238	0.07012	0.00113
12500	0.63270	0.03394	0.05784	0.00082
15000	0.44972	0.02157	0.04581	0.00100
17500	0.44924	0.02296	0.04332	0.00060
20000	0.41018	0.02141	0.03868	0.00054
22500	0.31794	0.01427	0.03457	0.00049
25000	0.26285	0.01070	0.03056	0.00061
27500	0.21871	0.00805	0.02712	0.00052
30000	0.19578	0.00702	0.02470	0.00041
32500	0.17387	0.00561	0.02314	0.00042
35000	0.15051	0.00394	0.02187	0.00035
37500	0.13306	0.00286	0.02053	0.00036
40000	0.13759	0.00390	0.01940	0.00032
42500	0.11622	0.00228	0.01837	0.00031
45000	0.11086	0.00220	0.01748	0.00029
47500	0.10621	0.00212	0.01674	0.00027

Table 10. Heston PINN training-loss history.

Epoch	Total	PDE	IC	BC
0	251692.2969	0.03398	4686.0298	40966.3320
2500	2825.07788	188.86124	82.19197	22.90919
5000	2080.96826	116.06000	34.98023	114.11321
7500	492.30597	32.44241	13.90004	5.77630
10000	244.53905	12.81886	9.48032	4.30945
12500	306.85504	20.54513	7.70190	4.87695
15000	421.74133	23.56755	11.81930	13.57456
17500	1769.61401	92.72594	23.29924	121.87243
20000	97.64471	3.14415	3.17083	6.89899
22500	320.85370	18.72681	2.79603	21.12504
25000	72.54399	1.39077	2.41323	6.90079
27500	77.87282	2.30286	2.48623	5.99639
30000	41.28610	3.38855	0.55816	0.36379
32500	27.03453	1.55174	0.35608	1.59127
35000	13.38450	0.33137	0.50636	1.00144
37500	9.27743	0.32270	0.34799	0.51411
40000	4.70522	0.18073	0.21513	0.14932
42500	2.72396	0.06335	0.12588	0.16633
45000	1.38333	0.04318	0.08533	0.01965
47500	1.04974	0.03381	0.07039	0.00156

Table 11. Inverse PINN training and calibration history with evolving parameter estimates κ , θ , ξ , ρ . After epoch 25,500 all reported quantities are stationary to the displayed precision; the plateau is indicated by an ellipsis row, with the final epoch (29,999) shown to mark the end of training.

Epoch	Total	PDE	IC	BC	Data	Feller	κ	θ	ξ	ρ
0	5042358.0000	10.725791	4134.6187	28.860662	5000.6528	0.002455	0.999	0.1001	0.500	−0.001
1500	4059.0977	87.752625	29.4920	0.367506	2.0073	0.000000	1.620	0.0422	0.163	0.793
3000	1085.7815	11.841378	7.7701	0.067702	0.7709	0.000000	1.578	0.0357	0.133	0.888
4500	1028.3889	5.079397	3.6259	0.043186	0.8903	0.000000	1.536	0.0344	0.123	0.921
6000	283.0856	5.284629	2.4379	0.042856	0.1528	0.000000	1.504	0.0345	0.118	0.940
7500	1080.6104	21.654047	3.2561	0.109399	0.6144	0.000000	1.485	0.0344	0.115	0.955
9000	141.1073	2.657735	1.3538	0.017838	0.0743	0.000000	1.484	0.0337	0.112	0.964
10500	357.5637	5.206030	1.4301	0.059183	0.2388	0.000000	1.469	0.0334	0.110	0.973
12000	82.9516	1.525354	0.9522	0.021463	0.0428	0.000000	1.480	0.0320	0.109	0.978
13500	144.8452	3.712763	0.8913	0.042495	0.0615	0.000000	1.493	0.0307	0.107	0.983
15000	86.0975	1.212989	0.7499	0.004642	0.0543	0.000000	1.501	0.0300	0.107	0.986
16500	42.8948	1.066067	0.6579	0.002672	0.0150	0.000000	1.514	0.0293	0.107	0.988
18000	28.8773	0.705716	0.4965	0.002691	0.0098	0.000000	1.536	0.0282	0.106	0.991
19500	36.3960	1.234485	0.5088	0.004252	0.0066	0.000000	1.548	0.0278	0.106	0.992
21000	25.2769	0.750561	0.4207	0.003757	0.0060	0.000000	1.562	0.0273	0.105	0.993
22500	19.7579	0.587920	0.4024	0.002703	0.0040	0.000000	1.574	0.0269	0.105	0.994
24000	17.8799	0.548446	0.3758	0.002470	0.0031	0.000000	1.584	0.0267	0.105	0.994
25500	17.3094	0.533012	0.3654	0.002380	0.0030	0.000000	1.588	0.0266	0.104	0.994
... stationary plateau ...										
29999	17.3094	0.533012	0.3654	0.002380	0.0030	0.000000	1.588	0.0266	0.104	0.994

Notes for Tables 9–11. **(BS)** 50000 epochs in 12 min 8 s; 6250 points (5000 interior, 1000 IC, 250 BC); final total loss 0.102625. Adam with $\eta_0 = 5 \times 10^{-4}$, cosine-annealing (Loshchilov and Hutter 2017) to $\eta_{\min} = 10^{-6}$, gradient-norm clipping at 1.0 (Pascanu et al. 2013), and loss weights $(\lambda_{\text{PDE}}, \lambda_{\text{IC}}, \lambda_{\text{BC}}) = (10, 5, 5)$. **(Heston)** 50000 epochs in 24 min 40 s; 12500 points (10000 interior, 2000 IC, 250 lower- S BC, 250 upper- S BC); final total loss 0.921236. Same optimizer schedule with $\eta_0 = 5 \times 10^{-3}$, $(\lambda_{\text{PDE}}, \lambda_{\text{IC}}, \lambda_{\text{BC}}) = (10, 10, 5)$, and a 3-layer, 256-neuron **tanh** network. **(Inverse)** 30000 epochs in 16 min 17 s (25000 Adam + 5000 L-BFGS (D. C. Liu and Nocedal 1989)); final losses: total 17.3094, PDE 0.533012, data 0.0030; final calibrated $\kappa = 1.588$, $\theta = 0.0266$, $\xi = 0.104$, $\rho = 0.994$, with true generating values $\kappa = 2.0$, $\theta = 0.04$, $\xi = 0.3$, $\rho = -0.7$.

A.3 Black-Scholes PINN

The Black-Scholes hyperparameter study consists of four 10,000-epoch sweeps on the standard collocation grid: architecture, Adam learning rate, loss weights, and epoch budget. The ranked summaries and training curves tell the same story. **tanh** dominates **softplus**; at least 128 neurons are needed for sub- 10^{-2} residuals; $\eta_0 = 5 \times 10^{-4}$ sits in the stable region before large learning rates begin to oscillate; PDE-heavy loss weights perform best; and the residual is still decaying at 10,000 epochs, motivating the final 50,000-epoch run.

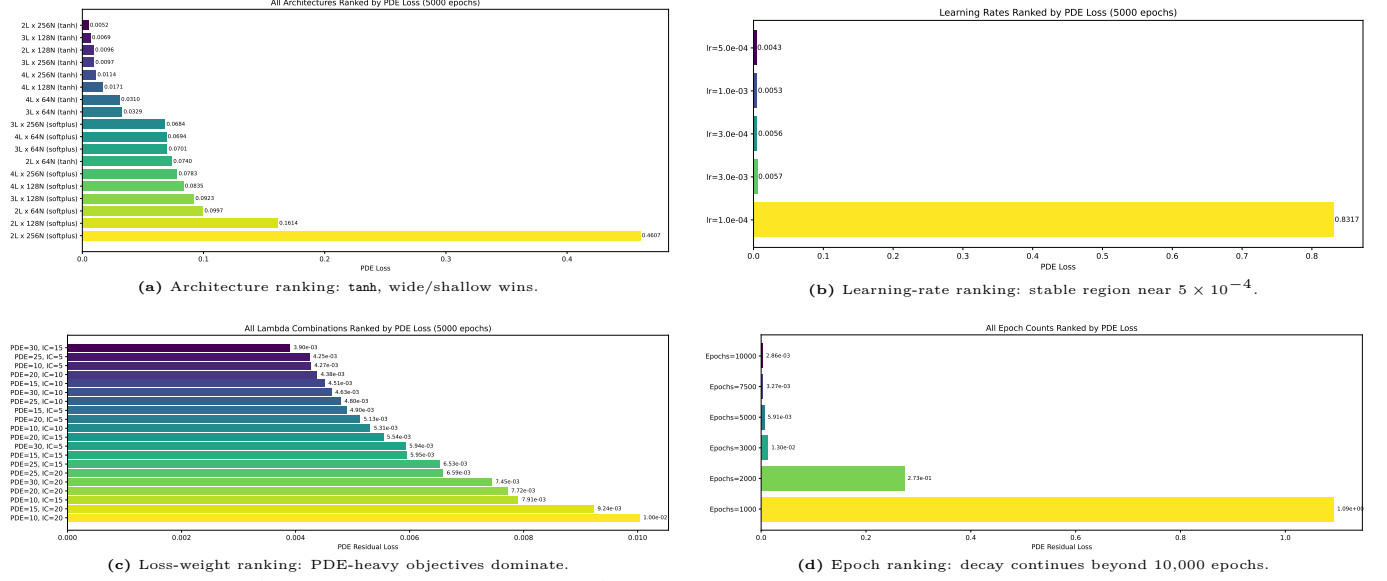


Figure 10. Ranked Black-Scholes hyperparameter sweeps. The four panels summarize the architecture, learning-rate, loss-weight, and epoch-budget experiments used to select the final Black-Scholes PINN configuration.

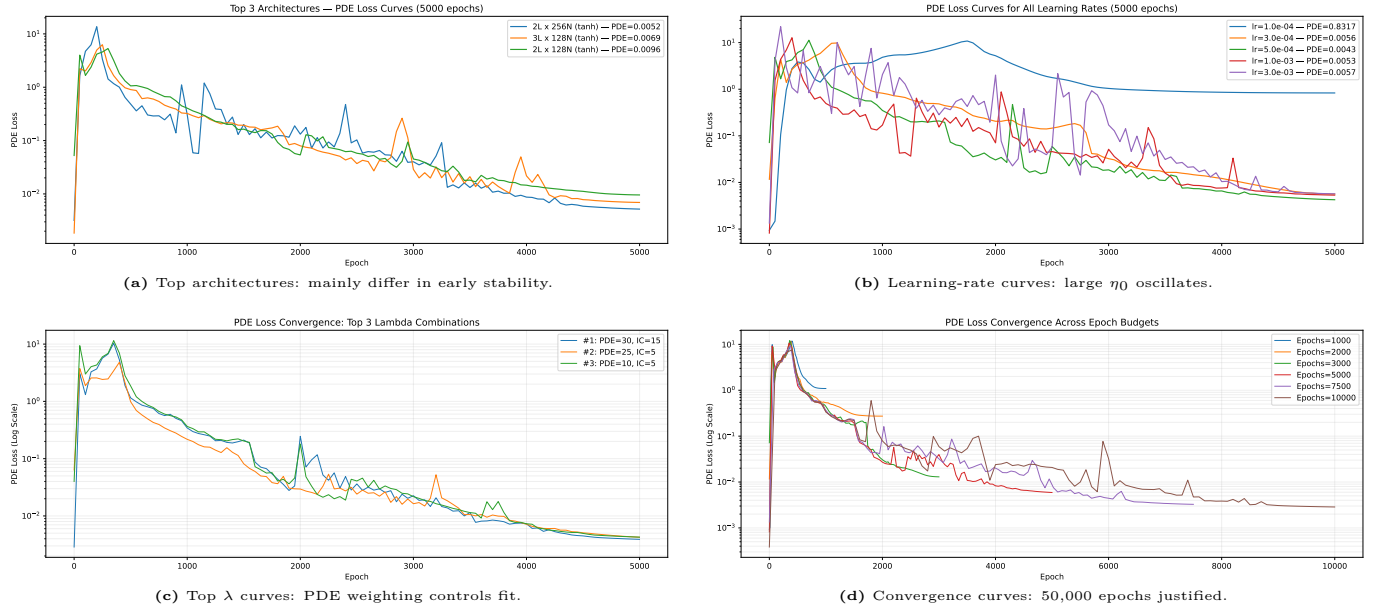


Figure 11. Training-curve comparison for the main Black-Scholes hyperparameter experiments. The four panels show the effects of architecture choice, learning rate, loss-weight selection, and training duration on convergence behavior.

A.3.1 Per-component loss decomposition

The loss decompositions compare the final 50,000-epoch forward PINN runs. Black-Scholes converges smoothly: the total loss falls from $\approx 2.2 \times 10^4$ to 0.103, the PDE residual settles near 2×10^{-3} , the IC loss dominates because of the kinked payoff, and the BC loss becomes negligible early. Heston is noisier: the larger (S, v, τ) state space, upper- S boundary, and mixed derivative $\partial^2 V / \partial S \partial v$ produce transient spikes before convergence to total loss ≈ 0.92 .

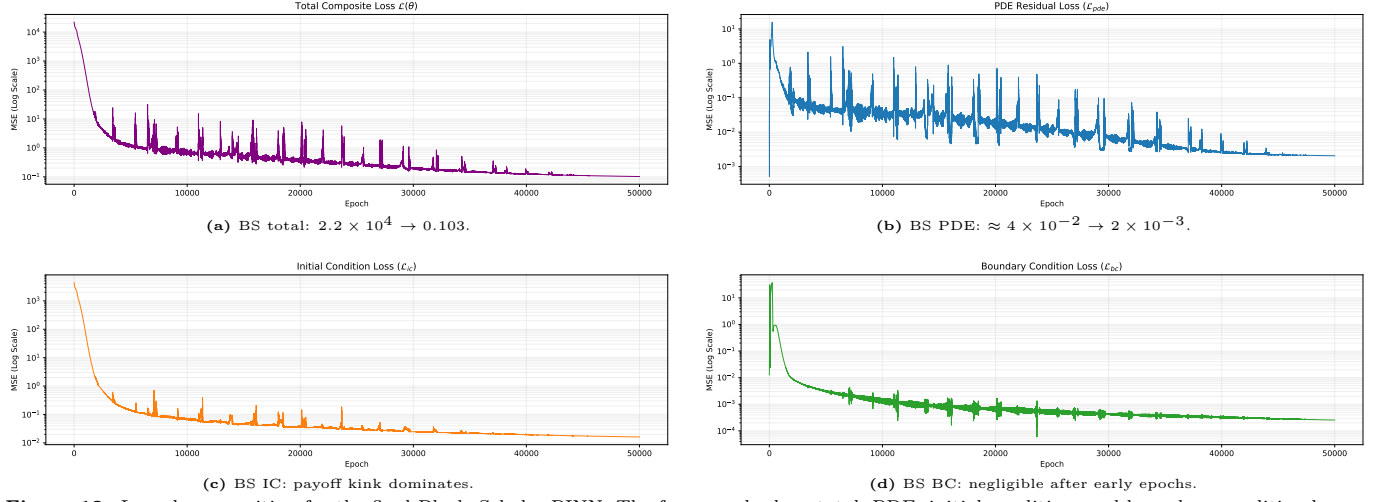


Figure 12. Loss decomposition for the final Black-Scholes PINN. The four panels show total, PDE, initial-condition, and boundary-condition losses.

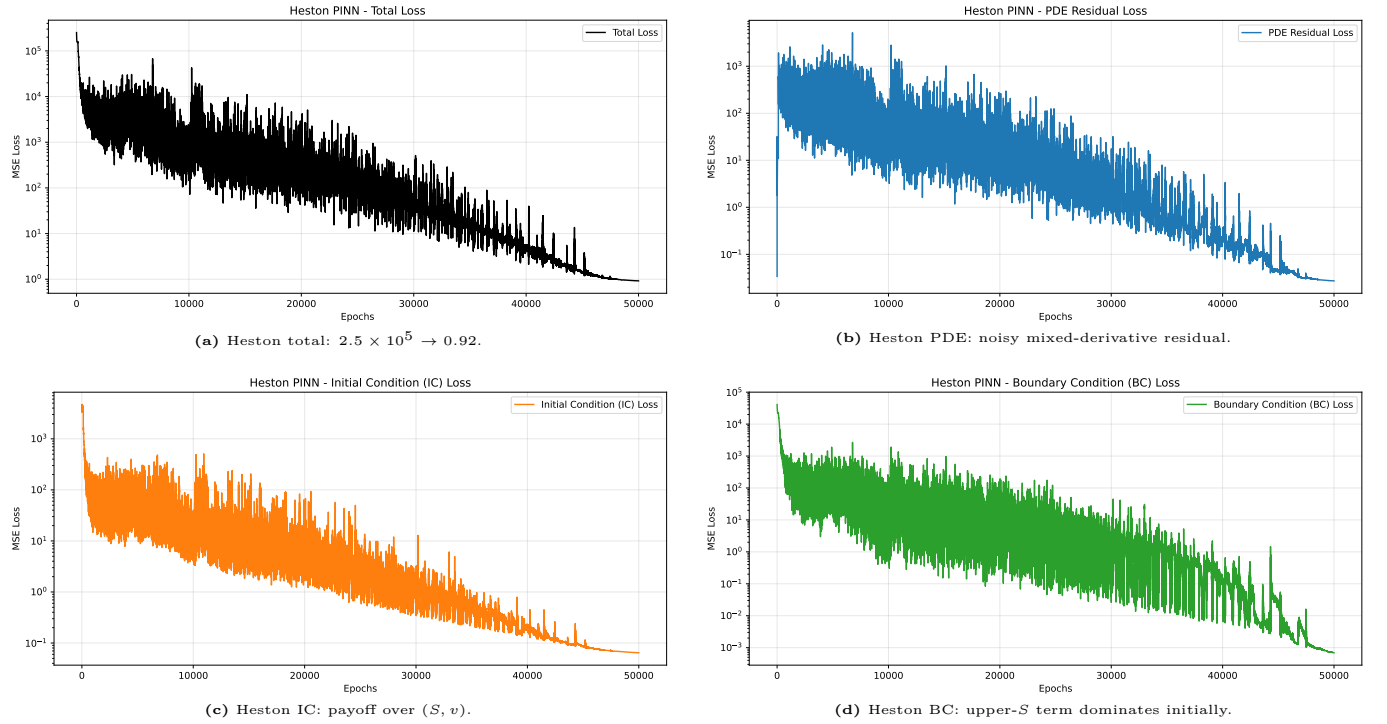
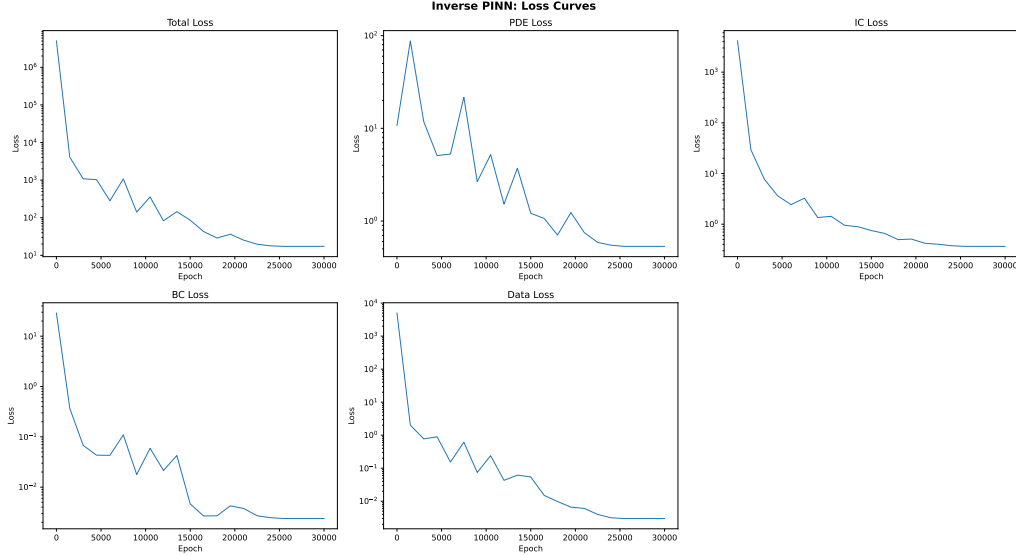


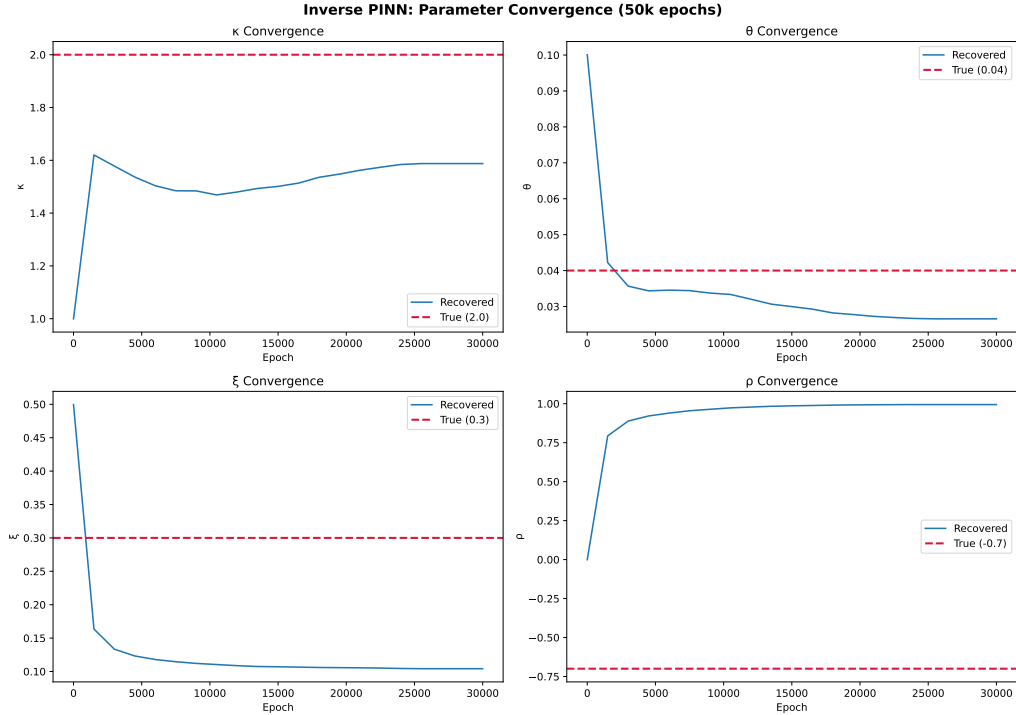
Figure 13. Loss decomposition for the final Heston PINN. The four panels show total, PDE, initial-condition, and boundary-condition losses.

A.4 Inverse PINN

The inverse PINN was trained on a synthetic Heston dataset of 200 noisy price observations spread across the (S, v, τ) domain, with the four Heston parameters reparameterized via softplus/tanh (Listing 2). Optimization used Adam for 25,000 epochs followed by 5,000 L-BFGS refinement steps (D. C. Liu and Nocedal 1989), with $(\lambda_{\text{PDE}}, \lambda_{\text{IC}}, \lambda_{\text{BC}}, \lambda_{\text{data}}, \lambda_{\text{Feller}}) = (20, 10, 5, 1000, 0.1)$, NN learning rate $\eta_0 = 2 \times 10^{-3}$, and Heston-parameter learning rate $\eta_{\text{Heston}} = 10^{-3}$. The loss panel shows successful optimization of the pricing objective: the data loss drops below 10^{-2} by epoch $\sim 20,000$ and the Feller penalty remains at machine zero throughout. The parameter panel shows the opposite conclusion structurally: the final calibrated values $(\kappa, \theta, \xi, \rho) = (1.588, 0.0266, 0.104, 0.994)$ differ sharply from the data-generating truth $(2.0, 0.04, 0.3, -0.7)$ in Table 11, with ρ converging to the wrong sign.



(a) Inverse losses: the data loss falls below 10^{-2} and the Feller penalty stays at zero.



(b) Parameter paths: ξ collapses toward ≈ 0.1 and ρ converges to $+0.994$ instead of the true -0.7 .

Figure 14. Inverse PINN calibration diagnostics. The top panel shows the per-component loss decomposition over Adam and L-BFGS training; the bottom panel shows the corresponding Heston-parameter trajectories. Together, the panels show the key inverse-calibration result: the PINN fits prices well while failing to recover the true structural parameters.

A.4.1 Final calibration report

The final calibration report collects the main diagnostics of the inverse Heston experiment in a single figure. Taken at face value, the pricing fit looks highly successful: predicted prices track the synthetic market prices closely, the error distribution is tightly centered around zero, and the maturity-slice comparison at $\tau \approx 0.5$ shows that the learned pricing function reproduces the observed option-price curve well across asset prices. In other words, as a differentiable pricing surrogate, the inverse PINN performs very well.

The structural conclusion is different. The parameter comparison shows that accurate price fitting does not imply correct recovery of the data-generating Heston parameters. The final estimates $(\kappa, \theta, \xi, \rho) = (1.588, 0.0266, 0.104, 0.994)$ remain biased relative to the true values $(2.0, 0.04, 0.3, -0.7)$, most notably because ρ is recovered with the wrong sign. This is the clearest illustration of the identifiability problem in the report: in the Heston PDE, ρ enters only through the mixed term $\rho \xi v S \partial^2 V / \partial S \partial v$, so it is identified mainly through leverage-skew information in the option surface. With only 200 noisy observations spread across a broad variance domain, that skew signal is too weak to force the optimizer onto the true negative-correlation branch.

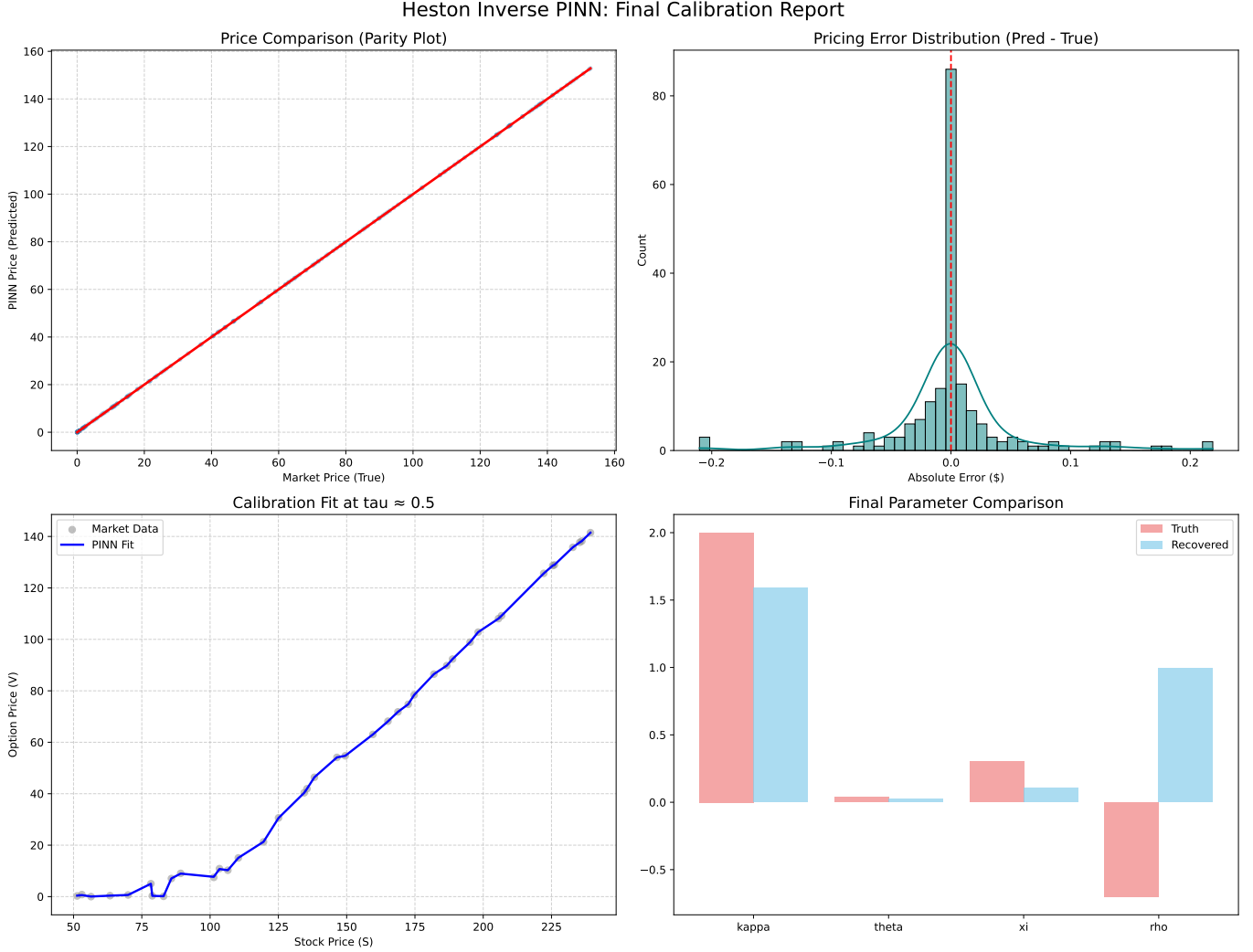


Figure 15. Final inverse Heston PINN calibration diagnostics. The figure shows strong price fit but weak structural recovery: predicted prices match the synthetic market prices closely, while the recovered Heston parameters remain biased and ρ converges to the wrong sign.